ĐẠI SỐ TUYẾN TÍNH

VÀ CÁC ỨNG DỤNG

CHƯƠNG 8

TÍNH TRỰC GIAO

(Orthogonality)

Bản dịch tiếng Việt

Nguyên tác: **W. Keith Nicholson**

Linear Algebra with Applications, Open Edition

Version 2023 — Revision A

Giấy phép và ghi công (License & Attribution)

Chương này được dịch (adapted) từ nguyên tác *Linear Algebra with Applications* của W. Keith Nicholson và Lyryx Learning Inc. Xem bản gốc miễn phí tại `https://lyryx.com/linear-algebra-applications/`.

Tóm tắt thay đổi: Toàn bộ nội dung lý thuyết Chương 8 được dịch từ tiếng Anh sang tiếng Việt. Bổ sung bảng đối chiếu thuật ngữ Anh–Việt và ghi chú cho người học. Các ký hiệu toán học, công thức và ví dụ giữ nguyên theo bản gốc.

Giấy phép: Creative Commons Attribution-NonCommercial-ShareAlike 4.0 (CC BY-NC-SA 4.0) — `http://creativecommons.org/licenses/by-nc-sa/4.0/`. Bản dịch này được phát hành theo cùng giấy phép với bản gốc.

Trích dẫn nguyên tác: Nicholson, W. Keith. *Linear Algebra with Applications*, Version 2023 – Revision A. Vretta-Lyryx Inc., Calgary, Alberta, Canada, July 1, 2023.

Đọc trước khi bắt đầu — Chương 8 là chương ứng dụng quan trọng nhất

Nếu bạn chỉ có thời gian học kỹ *một* chương trong sách này để phục vụ công việc IT / machine learning / bảo mật, hãy chọn Chương 8.
Toàn bộ chương xoay quanh một ý tưởng: **trực giao làm mọi thứ trở nên dễ**. Bạn đã thấy điều này ở Mục 5.3 (tọa độ tính bằng tích vô hướng) và Mục 5.6 (bình phương tối thiểu = phép chiếu). Chương 8 khai thác triệt để ý tưởng đó.
Bản đồ chương:

- **8.1–8.2:** Công cụ nền — Gram–Schmidt, phép chiếu trực giao, chéo hóa trực giao. *Bắt buộc phải nắm.*
- **8.3–8.5:** Ma trận xác định dương, phân rã QR, tính trị riêng bằng máy tính. *Đây là phần "làm sao máy tính thực sự tính được".*
- **8.6: SVD** — phân rã giá trị kỳ dị. Nếu phải chọn *một* mục, chọn mục này. SVD là nền của PCA, nén ảnh, hệ gợi ý, và nghịch đảo giả.
- **8.7:** Ma trận phức — nền của xử lý tín hiệu và cơ học lượng tử.
- **8.8: Mã tuyến tính trên trường hữu hạn** — mục quan trọng nhất cho ngành bảo mật. Đây là toán đằng sau mã sửa lỗi, QR code, RAID, và nhiều hệ mật mã.
- **8.9–8.11:** Dạng toàn phương, tối ưu có ràng buộc, và PCA.

Lời khuyên học: chương này dài. Đừng cố đọc một mạch. Học 8.1–8.2 thật kỹ (chúng là nền), rồi nhảy tới mục nào bạn cần. Các mục 8.8–8.11 gần như độc lập với nhau.

Bảng đối chiếu thuật ngữ Anh–Việt (Chương 8)

Tiếng Anh	Tiếng Việt	Giải thích ngắn
orthogonal	trực giao	Vuông góc: $\mathbf{x} \cdot \mathbf{y} = 0$
orthonormal	trực chuẩn	Trực giao + mọi vector có độ dài 1
orthogonal complement	phần bù trực giao	$U^\perp$: mọi vector vuông góc với U
Gram–Schmidt algorithm	thuật toán Gram–Schmidt	Biến cơ sở bất kỳ thành cơ sở trực giao
projection	phép chiếu	$\text{proj}_U \mathbf{x}$: điểm trong U gần $\mathbf{x}$ nhất
Projection Theorem	Định lý chiếu	$\mathbf{x} - \mathbf{p} \in U^\perp$ và $\mathbf{p}$ gần nhất
orthogonal matrix	ma trận trực giao	$P^T P = I$, tức $P^{-1} = P^T$
orthogonally diagonalizable	chéo hóa trực giao được	$P^T A P$ chéo với P trực giao
Principal Axis Theorem	Định lý trục chính	A đối xứng $\iff$ chéo hóa trực giao được
Spectral Theorem	Định lý phổ	Tên gọi khác của Định lý trục chính
spectrum	phổ	Tập các trị riêng
symmetric matrix	ma trận đối xứng	$A^T = A$
positive definite	xác định dương	$\mathbf{x}^T A \mathbf{x} > 0$ với mọi $\mathbf{x} \neq \mathbf{0}$
Cholesky factorization	phân rã Cholesky	$A = U^T U$ với U tam giác trên
QR-factorization	phân rã QR	$A = QR$, Q trực giao, R tam giác trên
QR-algorithm	thuật toán QR	Cách máy tính tìm trị riêng
singular value	giá trị kỳ dị	$\sigma_i = \sqrt{\lambda_i}$ của $A^T A$
Singular Value Decomposition (SVD)	phân rã giá trị kỳ dị	$A = U \Sigma V^T$
fundamental subspaces	bốn không gian cơ bản	row A, col A, null A, null A^T
polar decomposition	phân rã cực	$A = GQ$: ”giãn rồi quay”
pseudoinverse	nghịch đảo giả	A^+: nghịch đảo tổng quát hóa
conjugate	liên hợp	$\overline{z}$ của số phức z
conjugate transpose	chuyển vị liên hợp	$A^H = \overline{A}^T$
hermitian matrix	ma trận Hermite	$A^H = A$ (phiên bản phức của đối xứng)
unitary matrix	ma trận unita	$U^H U = I$ (phiên bản phức của trực giao)
normal matrix	ma trận chuẩn tắc	$AA^H = A^H A$
finite field	trường hữu hạn	Trường có hữu hạn phần tử, ví dụ $\mathbb{Z}_p$
linear code	mã tuyến tính	Không gian con dùng để sửa lỗi
codeword	từ mã	Phần tử của mã
Hamming weight	trọng số Hamming	Số tọa độ khác 0
Hamming distance	khoảng cách Hamming	Số vị trí khác nhau
generator matrix	ma trận sinh	Ma trận có các hàng là cơ sở của mã
parity-check matrix	ma trận kiểm tra chẵn lẻ	H với null H = mã
syndrome	hội chứng	$H\mathbf{x}$: dùng để định vị lỗi
quadratic form	dạng toàn phương	$q(\mathbf{x}) = \mathbf{x}^T A \mathbf{x}$
congruent matrices	ma trận đồng dư	$B = U^T A U$ với U khả nghịch
index / rank of a form	chỉ số / hạng của dạng	Số hệ số dương / số hệ số khác 0
constrained optimization	tối ưu có ràng buộc	Cực trị của $q(\mathbf{x})$ với $\|\mathbf{x}\| = 1$

Tiếng Anh	Tiếng Việt	Giải thích ngắn
principal component	thành phần chính	Hướng giữ nhiều phương sai nhất

8.1 Phần bù trực giao và phép chiếu

Bổ đề 8.1.1 — Bổ đề trực giao (Orthogonal Lemma)

Cho $\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$ là tập trực giao trong $\mathbb{R}^n$. Với $\mathbf{x}$ trong $\mathbb{R}^n$, đặt

$$\mathbf{f}_{m+1} = \mathbf{x} - \frac{\mathbf{x}\cdot\mathbf{f}_1}{\|\mathbf{f}_1\|^2}\mathbf{f}_1 - \frac{\mathbf{x}\cdot\mathbf{f}_2}{\|\mathbf{f}_2\|^2}\mathbf{f}_2 - \cdots - \frac{\mathbf{x}\cdot\mathbf{f}_m}{\|\mathbf{f}_m\|^2}\mathbf{f}_m$$

Khi đó:

1. $\mathbf{f}_{m+1}\cdot\mathbf{f}_k = 0$ với $k = 1, 2, \ldots, m$.
2. Nếu $\mathbf{x}$ *không* nằm trong span$\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$, thì $\mathbf{f}_{m+1} \neq \mathbf{0}$ và $\{\mathbf{f}_1, \ldots, \mathbf{f}_m, \mathbf{f}_{m+1}\}$ là tập trực giao.

Chứng minh. Đặt $t_i = (\mathbf{x}\cdot\mathbf{f}_i)/\|\mathbf{f}_i\|^2$. Với $1 \le k \le m$:

$$\begin{aligned}\mathbf{f}_{m+1}\cdot\mathbf{f}_k &= (\mathbf{x} - t_1\mathbf{f}_1 - \cdots - t_m\mathbf{f}_m)\cdot\mathbf{f}_k \\ &= \mathbf{x}\cdot\mathbf{f}_k - t_k(\mathbf{f}_k\cdot\mathbf{f}_k) \quad \text{(các số hạng khác triệt tiêu)} \\ &= \mathbf{x}\cdot\mathbf{f}_k - t_k\|\mathbf{f}_k\|^2 = 0\end{aligned}$$

Đó là (1); (2) suy ra vì $\mathbf{f}_{m+1} \neq \mathbf{0}$ khi $\mathbf{x}$ nằm ngoài span. ■

Định lý 8.1.1

Cho U là không gian con của $\mathbb{R}^n$.

1. Mọi tập con trực giao $\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$ trong U đều là tập con của một cơ sở trực giao của U.
2. U có một cơ sở trực giao.

Thuật toán Gram–Schmidt

Bổ đề trực giao cho ta một quy trình biến *bất kỳ* cơ sở $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ của U thành cơ sở trực giao $\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$.

Định lý 8.1.2 — Thuật toán trực giao hóa Gram–Schmidt

Nếu $\{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ là cơ sở bất kỳ của không gian con U của $\mathbb{R}^n$, xây dựng $\mathbf{f}_1, \ldots, \mathbf{f}_m$ lần lượt như sau:

$$\begin{aligned}\mathbf{f}_1 &= \mathbf{x}_1 \\ \mathbf{f}_2 &= \mathbf{x}_2 - \frac{\mathbf{x}_2\cdot\mathbf{f}_1}{\|\mathbf{f}_1\|^2}\mathbf{f}_1 \\ \mathbf{f}_3 &= \mathbf{x}_3 - \frac{\mathbf{x}_3\cdot\mathbf{f}_1}{\|\mathbf{f}_1\|^2}\mathbf{f}_1 - \frac{\mathbf{x}_3\cdot\mathbf{f}_2}{\|\mathbf{f}_2\|^2}\mathbf{f}_2 \\ &\vdots \\ \mathbf{f}_k &= \mathbf{x}_k - \frac{\mathbf{x}_k\cdot\mathbf{f}_1}{\|\mathbf{f}_1\|^2}\mathbf{f}_1 - \frac{\mathbf{x}_k\cdot\mathbf{f}_2}{\|\mathbf{f}_2\|^2}\mathbf{f}_2 - \cdots - \frac{\mathbf{x}_k\cdot\mathbf{f}_{k-1}}{\|\mathbf{f}_{k-1}\|^2}\mathbf{f}_{k-1}\end{aligned}$$

với $k = 2, 3, \ldots, m$. Khi đó:

1. $\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$ là cơ sở *trực giao* của U.

2. $\text{span}\{\mathbf{f}_1, \ldots, \mathbf{f}_k\} = \text{span}\{\mathbf{x}_1, \ldots, \mathbf{x}_k\}$ với mọi k.

Ghi chú cho người học — Gram–Schmidt nói gì bằng lời

Công thức trông cồng kềnh nhưng ý tưởng rất đơn giản, chỉ một câu:

Lấy vector mới, trừ đi hình chiếu của nó lên tất cả những gì đã dựng được.

Phần còn lại tất nhiên vuông góc với mọi thứ trước đó — vì ta đã "vắt kiệt" mọi thành phần song song.
Mẹo tính tay: sau khi tính được mỗi $\mathbf{f}_k$, bạn được phép *nhân nó với một số khác 0 bất kỳ* để khử phân số — các $\mathbf{f}$ sau đó không đổi. Ví dụ 8.1.1 dùng mẹo này.
Cảnh báo khi lập trình: thuật toán viết như trên (gọi là *classical Gram–Schmidt*) rất **kém ổn định số học** — sai số làm tròn tích lũy khiến các vector kết quả mất tính trực giao. Trong thực tế hãy dùng:

- *Modified Gram–Schmidt*: trừ hình chiếu *ngay lập tức* sau mỗi bước thay vì cộng dồn — cùng công thức toán học nhưng ổn định hơn nhiều.
- *Householder reflections* — cách chuẩn trong LAPACK; `numpy.linalg.qr` dùng cách này.

Gram–Schmidt cổ điển vẫn quan trọng vì nó *giải thích* tại sao phân rã QR tồn tại (Mục 8.4) — nhưng đừng dùng nó để tính toán thật.

Ví dụ 8.1.1

Tìm cơ sở trực giao của không gian hàng của $A = \begin{bmatrix} 1 & 1 & -1 & -1 \\ 3 & 2 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$.

Lời giải. Gọi $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ là các hàng, chúng độc lập tuyến tính. Lấy $\mathbf{f}_1 = \mathbf{x}_1 = (1, 1, -1, -1)$. Thuật toán cho

$$\mathbf{f}_2 = \mathbf{x}_2 - \tfrac{4}{4}\mathbf{f}_1 = (3, 2, 0, 1) - (1, 1, -1, -1) = (2, 1, 1, 2)$$

$$\mathbf{f}_3 = \mathbf{x}_3 - \tfrac{0}{4}\mathbf{f}_1 - \tfrac{3}{10}\mathbf{f}_2 = \tfrac{1}{10}(4, -3, 7, -6)$$

Vậy $\{(1, 1, -1, -1),\ (2, 1, 1, 2),\ \frac{1}{10}(4, -3, 7, -6)\}$ là cơ sở trực giao. Khử phân số, $\{(1, 1, -1, -1),\ (2, 1, 1, 2),\ (4, -3, 7, -6)\}$ cũng là cơ sở trực giao của row A.

Ghi chú lịch sử

Erhardt Schmidt (1876–1959) là nhà toán học Đức, học trò của David Hilbert, sau này phát triển lý thuyết không gian Hilbert. Ông mô tả thuật toán này lần đầu năm 1907. Jörgen Pederson Gram (1850–1916) là một chuyên gia tính toán bảo hiểm người Đan Mạch.

Phần bù trực giao và phép chiếu

Định nghĩa 8.1 — Phần bù trực giao

Nếu U là không gian con của $\mathbb{R}^n$, **phần bù trực giao** $U^\perp$ (đọc là "U-perp") được định nghĩa bởi

$$U^\perp = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{x} \cdot \mathbf{y} = 0 \text{ với mọi } \mathbf{y} \in U\}$$

Bổ đề 8.1.2

Cho U là không gian con của $\mathbb{R}^n$.

1. $U^\perp$ là không gian con của $\mathbb{R}^n$.
2. $\{\mathbf{0}\}^\perp = \mathbb{R}^n$ và $(\mathbb{R}^n)^\perp = \{\mathbf{0}\}$.
3. Nếu $U = \text{span}\{\mathbf{x}_1, \ldots, \mathbf{x}_k\}$, thì $U^\perp = \{\mathbf{x} \mid \mathbf{x} \cdot \mathbf{x}_i = 0 \text{ với } i = 1, \ldots, k\}$.

Chứng minh (3). Nếu $\mathbf{x} \in U^\perp$ thì $\mathbf{x} \cdot \mathbf{x}_i = 0$ vì mỗi $\mathbf{x}_i \in U$. Ngược lại, giả sử $\mathbf{x} \cdot \mathbf{x}_i = 0$ với mọi i. Với $\mathbf{y} = r_1\mathbf{x}_1 + \cdots + r_k\mathbf{x}_k$ bất kỳ trong U:

$$\mathbf{x} \cdot \mathbf{y} = r_1(\mathbf{x} \cdot \mathbf{x}_1) + \cdots + r_k(\mathbf{x} \cdot \mathbf{x}_k) = 0$$

■

Điểm quan trọng của Bổ đề 8.1.2(3)

Đây là mẹo tính toán then chốt: để tìm $U^\perp$, bạn *không* cần kiểm tra vô hạn vector trong U — chỉ cần bắt vuông góc với các *vector sinh*. Điều đó biến bài toán thành một hệ phương trình thuần nhất.
Nói theo ngôn ngữ Mục 5.4: nếu A là ma trận có các hàng là $\mathbf{x}_1, \ldots, \mathbf{x}_k$, thì

$$U^\perp = \text{null}(A)$$

Vậy **phần bù trực giao của không gian hàng chính là không gian nghiệm**. Đây là một trong bốn quan hệ cơ bản mà Mục 8.6.2 sẽ hệ thống hóa.

Ví dụ 8.1.2

Tìm $U^\perp$ nếu $U = \text{span}\{(1, -1, 2, 0),\ (1, 0, -2, 3)\}$ trong $\mathbb{R}^4$.
Lời giải. Theo Bổ đề 8.1.2, $\mathbf{x} = (x, y, z, w)$ nằm trong $U^\perp$ khi và chỉ khi nó trực giao với cả hai vector sinh:

$$x - y + 2z = 0, \qquad x - 2z + 3w = 0$$

Khử Gauss cho $U^\perp = \text{span}\{(2, 4, 1, 0),\ (3, 3, 0, -1)\}$.

Định nghĩa 8.2 — Phép chiếu lên một không gian con

Cho U là không gian con của $\mathbb{R}^n$ với cơ sở trực giao $\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$. Với $\mathbf{x}$ trong $\mathbb{R}^n$, vector

$$\text{proj}_U \mathbf{x} = \frac{\mathbf{x} \cdot \mathbf{f}_1}{\|\mathbf{f}_1\|^2}\mathbf{f}_1 + \frac{\mathbf{x} \cdot \mathbf{f}_2}{\|\mathbf{f}_2\|^2}\mathbf{f}_2 + \cdots + \frac{\mathbf{x} \cdot \mathbf{f}_m}{\|\mathbf{f}_m\|^2}\mathbf{f}_m$$

gọi là **hình chiếu trực giao** của $\mathbf{x}$ lên U. Với $U = \{\mathbf{0}\}$, định nghĩa $\text{proj}_{\{\mathbf{0}\}} \mathbf{x} = \mathbf{0}$.

Định nghĩa này hợp lệ vì kết quả *không phụ thuộc* vào cơ sở trực giao được chọn. Thật vậy, nếu $\mathbf{p}$ và $\mathbf{p}'$ là hình chiếu tính theo hai cơ sở trực giao khác nhau, thì $\mathbf{p}-\mathbf{p}' = (\mathbf{x}-\mathbf{p}')-(\mathbf{x}-\mathbf{p})$ vừa nằm trong U vừa nằm trong $U^{\perp}$, nên nó trực giao với chính nó, tức bằng $\mathbf{0}$.

Định lý 8.1.3 — Định lý chiếu (Projection Theorem)

Nếu U là không gian con của $\mathbb{R}^n$ và $\mathbf{x} \in \mathbb{R}^n$, đặt $\mathbf{p} = \text{proj}_U \mathbf{x}$. Khi đó:

1. $\mathbf{p}$ nằm trong U và $\mathbf{x}-\mathbf{p}$ nằm trong $U^{\perp}$.
2. $\mathbf{p}$ là vector trong U **gần x nhất**:

$$\|\mathbf{x}-\mathbf{p}\| < \|\mathbf{x}-\mathbf{y}\| \qquad \text{với mọi } \mathbf{y} \in U,\ \mathbf{y} \neq \mathbf{p}$$

Chứng minh (2). Viết $\mathbf{x}-\mathbf{y} = (\mathbf{x}-\mathbf{p})+(\mathbf{p}-\mathbf{y})$. Khi đó $\mathbf{p}-\mathbf{y}$ nằm trong U, nên trực giao với $\mathbf{x}-\mathbf{p}$ theo (1). Định lý Pythagoras cho

$$\|\mathbf{x}-\mathbf{y}\|^2 = \|\mathbf{x}-\mathbf{p}\|^2 + \|\mathbf{p}-\mathbf{y}\|^2 > \|\mathbf{x}-\mathbf{p}\|^2$$

vì $\mathbf{p}-\mathbf{y} \neq \mathbf{0}$. ■

Ví dụ 8.1.3

Cho $U = \text{span}\{\mathbf{x}_1, \mathbf{x}_2\}$ trong $\mathbb{R}^4$ với $\mathbf{x}_1 = (1,1,0,1)$, $\mathbf{x}_2 = (0,1,1,2)$. Nếu $\mathbf{x} = (3,-1,0,2)$, tìm vector trong U gần $\mathbf{x}$ nhất và phân tích $\mathbf{x}$ thành tổng của một vector trong U và một vector trực giao với U.

Lời giải. $\{\mathbf{x}_1, \mathbf{x}_2\}$ độc lập nhưng không trực giao. Gram–Schmidt cho $\mathbf{f}_1 = (1,1,0,1)$ và

$$\mathbf{f}_2 = \mathbf{x}_2 - \tfrac{3}{3}\mathbf{f}_1 = (-1,0,1,1)$$

Vậy

$$\mathbf{p} = \text{proj}_U \mathbf{x} = \tfrac{4}{3}\mathbf{f}_1 + \tfrac{-1}{3}\mathbf{f}_2 = \tfrac{1}{3}(5,4,-1,3)$$

Do đó $\mathbf{p}$ là vector trong U gần $\mathbf{x}$ nhất, và $\mathbf{x}-\mathbf{p} = \tfrac{1}{3}(4,-7,1,3)$ trực giao với mọi vector trong U. Phân tích cần tìm:

$$\mathbf{x} = \tfrac{1}{3}(5,4,-1,3) + \tfrac{1}{3}(4,-7,1,3)$$

Ví dụ 8.1.4

Tìm điểm trên mặt phẳng $2x+y-z=0$ gần điểm $(2,-1,-3)$ nhất.

Lời giải. Mặt phẳng là không gian con U gồm các điểm với $z = 2x+y$:

$$U = \{(s,t,2s+t) \mid s,t \in \mathbb{R}\} = \text{span}\{(0,1,1),\ (1,0,2)\}$$

Gram–Schmidt cho cơ sở trực giao $\mathbf{f}_1 = (0,1,1)$, $\mathbf{f}_2 = (1,-1,1)$. Vậy

$$\text{proj}_U \mathbf{x} = -2\mathbf{f}_1 + 0\mathbf{f}_2 = (0,-2,-2)$$

Điểm cần tìm là $(0,-2,-2)$.

Định lý 8.1.4

Cho U là không gian con cố định của $\mathbb{R}^n$, và $T : \mathbb{R}^n \to \mathbb{R}^n$ với $T(\mathbf{x}) = \text{proj}_U \mathbf{x}$. Khi đó:

1. T là toán tử tuyến tính.
2. $\text{im}\, T = U$ và $\ker T = U^{\perp}$.

3. $\dim U + \dim U^\perp = n$.

Chứng minh. Nếu $\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$ là cơ sở *trực chuẩn* của U, thì

$$T(\mathbf{x}) = (\mathbf{x} \cdot \mathbf{f}_1)\mathbf{f}_1 + \cdots + (\mathbf{x} \cdot \mathbf{f}_m)\mathbf{f}_m \tag{8.2}$$

T tuyến tính vì $(\mathbf{x}+\mathbf{y}) \cdot \mathbf{f}_i = \mathbf{x} \cdot \mathbf{f}_i + \mathbf{y} \cdot \mathbf{f}_i$ và $(r\mathbf{x}) \cdot \mathbf{f}_i = r(\mathbf{x} \cdot \mathbf{f}_i)$.

Với (2): $\operatorname{im} T \subseteq U$ theo (8.2). Nếu $\mathbf{x} \in U$ thì $\mathbf{x} = T(\mathbf{x})$ theo Định lý khai triển, nên $U \subseteq \operatorname{im} T$. Nếu $\mathbf{x} \in U^\perp$ thì $\mathbf{x} \cdot \mathbf{f}_i = 0$ với mọi i, nên $\mathbf{x} \in \ker T$. Ngược lại, Định lý 8.1.3 cho $\mathbf{x} - T(\mathbf{x}) \in U^\perp$, từ đó $\ker T \subseteq U^\perp$.

(3) suy ra từ (1), (2) và Định lý số chiều (Định lý 7.2.4). ■

Ghi chú cho người học — phép chiếu ở đâu trong thực tế

$\dim U + \dim U^\perp = n$ là một trong những công thức đẹp nhất của đại số tuyến tính: *mọi không gian con và phần bù trực giao của nó cộng lại đúng bằng cả không gian*. Mọi vector tách được *duy nhất* thành "phần trong U" cộng "phần vuông góc với U":

$$\mathbb{R}^n = U \oplus U^\perp$$

Ứng dụng:

- **Bình phương tối thiểu** (Mục 5.6): nghiệm tốt nhất chính là $\operatorname{proj}_{\operatorname{col} A}\mathbf{b}$. Định lý chiếu chính là lý do tại sao nghiệm đó tối ưu.
- **Nén dữ liệu / lọc nhiễu:** chiếu tín hiệu lên không gian con "tín hiệu thật"; phần trong $U^\perp$ là nhiễu bị loại bỏ.
- **PCA:** chiếu dữ liệu lên không gian con sinh bởi các thành phần chính.
- **Đồ họa:** bóng đổ, phản xạ, và "chiếu điểm lên mặt phẳng" đều là phép chiếu trực giao.
- **Gradient descent có ràng buộc:** sau mỗi bước, chiếu ngược về miền cho phép (projected gradient descent).

Ma trận chiếu: với $\{\mathbf{f}_i\}$ trực chuẩn xếp thành cột của Q, phép chiếu là nhân với $P = QQ^T$. Ma trận này thỏa $P^2 = P$ và $P^T = P$ — đó là định nghĩa *ma trận chiếu trực giao*. Trong code: `P = Q @ Q.T`, và bạn nên kiểm tra `np.allclose(P @ P, P)`.

8.2 Chéo hóa trực giao

Nhắc lại (Định lý 5.5.3): ma trận $n \times n$ A chéo hóa được khi và chỉ khi nó có n vector riêng độc lập tuyến tính. Vì các cơ sở *tốt nhất* của $\mathbb{R}^n$ là cơ sở trực giao, câu hỏi tự nhiên là: *ma trận nào có một cơ sở trực giao gồm vector riêng?* Câu trả lời hóa ra chính xác là **các ma trận đối xứng** — đó là kết quả chính của mục này.

Định lý 8.2.1

Các điều kiện sau tương đương với ma trận $n \times n$ P:

1. P khả nghịch và $P^{-1} = P^T$.
2. Các *hàng* của P trực chuẩn.

3. Các *cột* của P trực chuẩn.

Chứng minh. Điều kiện (1) tương đương $PP^T = I$. Gọi $\mathbf{x}_1, \ldots, \mathbf{x}_n$ là các hàng của P. Khi đó $\mathbf{x}_j^T$ là cột thứ j của P^T, nên phần tử (i,j) của PP^T là $\mathbf{x}_i \cdot \mathbf{x}_j$. Vậy $PP^T = I$ nghĩa là $\mathbf{x}_i \cdot \mathbf{x}_j = 0$ khi $i \neq j$ và $= 1$ khi $i = j$ — chính là (2). Tương tự cho (3). ■

Định nghĩa 8.3 — Ma trận trực giao

Ma trận $n \times n$ P gọi là **ma trận trực giao** (orthogonal matrix) nếu nó thỏa một (và do đó tất cả) các điều kiện của Định lý 8.2.1.

Ví dụ 8.2.1

Ma trận quay $\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ là trực giao với mọi góc θ.

Ghi chú cho người học — vì sao ma trận trực giao quan trọng

Ma trận trực giao có ba tính chất khiến chúng được dùng ở khắp nơi:
1. Nghịch đảo miễn phí: $P^{-1} = P^T$. Nghịch đảo một ma trận $n \times n$ thông thường tốn $O(n^3)$; với ma trận trực giao chỉ là hoán vị chỉ số — $O(1)$ nếu tính lười, $O(n^2)$ nếu sao chép.
2. Bảo toàn độ dài và góc: $\|P\mathbf{x}\| = \|\mathbf{x}\|$ và $(P\mathbf{x}) \cdot (P\mathbf{y}) = \mathbf{x} \cdot \mathbf{y}$. Về hình học, biến đổi trực giao chính là *quay* và *đối xứng* — chúng không làm méo hình.
3. Ổn định số học tuyệt vời: vì không khuếch đại độ dài, chúng *không khuếch đại sai số làm tròn*. Số điều kiện (condition number) của ma trận trực giao đúng bằng 1 — giá trị tốt nhất có thể.
Điểm 3 là lý do mọi thuật toán số học nghiêm túc đều xây trên phép biến đổi trực giao: QR (Mục 8.4), SVD (Mục 8.6), thuật toán QR tìm trị riêng (Mục 8.5), phản xạ Householder. Khi bạn gọi `numpy.linalg.lstsq` hay `scipy.linalg.svd`, bên trong là các phép quay và phản xạ trực giao.
Lưu ý tên gọi: theo (2) và (3), tên chính xác hơn phải là "ma trận *trực chuẩn*", vì các hàng/cột phải có độ dài 1 chứ không chỉ vuông góc. Nhưng "ma trận trực giao" là thuật ngữ chuẩn.

Ví dụ 8.2.2

Ma trận $\begin{bmatrix} 2 & 1 & 1 \\ -1 & 1 & 1 \\ 0 & -1 & 1 \end{bmatrix}$ có các hàng *trực giao* nhưng các cột thì không. Tuy nhiên, nếu chuẩn hóa các hàng, ma trận thu được

$$\begin{bmatrix} \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ \frac{-1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{-1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$$

là ma trận trực giao (nên các cột giờ cũng trực chuẩn).

Ví dụ 8.2.3

Nếu P và Q là ma trận trực giao, thì PQ cũng trực giao, và $P^{-1} = P^T$ cũng vậy.
Lời giải. $(PQ)^{-1} = Q^{-1}P^{-1} = Q^TP^T = (PQ)^T$. Tương tự $(P^{-1})^{-1} = P = (P^T)^T = (P^{-1})^T$.

Định nghĩa 8.4 — Ma trận chéo hóa trực giao được

Ma trận $n \times n$ A gọi là **chéo hóa trực giao được** (orthogonally diagonalizable) nếu tìm được ma trận trực giao P sao cho $P^{-1}AP = P^TAP$ là ma trận chéo.

Định lý 8.2.2 — Định lý trục chính (Principal Axes Theorem)

Các điều kiện sau tương đương với ma trận $n \times n$ A:

1. A có một tập trực chuẩn gồm n vector riêng.
2. A chéo hóa trực giao được.
3. A **đối xứng**.

Chứng minh.

(1) $\Leftrightarrow$ (2). Nếu $\mathbf{x}_1, \ldots, \mathbf{x}_n$ là vector riêng trực chuẩn, thì $P = [\,\mathbf{x}_1 \ \cdots \ \mathbf{x}_n\,]$ trực giao và $P^{-1}AP$ chéo. Ngược lại hiển nhiên.

(2) $\Rightarrow$ (3). Nếu $P^TAP = D$ chéo với $P^{-1} = P^T$, thì $A = PDP^T$. Vì $D^T = D$:

$$A^T = (PDP^T)^T = PD^TP^T = PDP^T = A$$

(3) $\Rightarrow$ (2). Quy nạp theo n. Với $n = 1$ hiển nhiên. Với $n > 1$, theo Định lý 5.5.7 lấy λ_1 là trị riêng *thực* của A, với $A\mathbf{x}_1 = \lambda_1\mathbf{x}_1$, $\|\mathbf{x}_1\| = 1$. Dùng Gram–Schmidt tìm cơ sở trực chuẩn $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ của $\mathbb{R}^n$, đặt $P_1 = [\,\mathbf{x}_1 \ \cdots \ \mathbf{x}_n\,]$. Khi đó P_1 trực giao và

$$P_1^TAP_1 = \begin{bmatrix} \lambda_1 & B \\ 0 & A_1 \end{bmatrix}$$

theo Bổ đề 5.5.2. Nhưng $P_1^TAP_1$ *đối xứng* (vì A đối xứng), nên $B = 0$ và A_1 đối xứng. Theo giả thiết quy nạp, tồn tại Q trực giao cỡ $(n-1)$ với $Q^TA_1Q = D_1$ chéo. Đặt $P_2 = \begin{bmatrix} 1 & 0 \\ 0 & Q \end{bmatrix}$ (trực giao), ta tính

$$(P_1P_2)^TA(P_1P_2) = \begin{bmatrix} 1 & 0 \\ 0 & Q^T \end{bmatrix}\begin{bmatrix} \lambda_1 & 0 \\ 0 & A_1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & Q \end{bmatrix} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & D_1 \end{bmatrix}$$

là ma trận chéo. ■

Tập vector riêng trực chuẩn của ma trận đối xứng A gọi là tập **trục chính** (principal axes) của A. Vì trị riêng của ma trận đối xứng thực đều là thực, Định lý 8.2.2 còn gọi là **định lý phổ thực** (real spectral theorem), và tập các trị riêng phân biệt gọi là **phổ** (spectrum) của ma trận.

Ghi chú cho người học — Định lý phổ là định lý quan trọng nhất của Chương 8

Định lý 8.2.2 phát biểu một điều mạnh đến mức đáng kinh ngạc:

Đối xứng $\iff$ *chéo hóa được bằng ma trận trực giao.*

Không phải "nếu", không phải "hầu hết" — mà là tương đương hoàn toàn. Mọi ma trận đối xứng thực đều viết được là

$$A = PDP^T \qquad (P \text{ trực giao},\ D \text{ chéo với các trị riêng thực})$$

Vì sao điều này quan trọng với ngành của bạn: ma trận đối xứng xuất hiện ở khắp nơi trong dữ liệu và tối ưu:

- **Ma trận hiệp phương sai** trong PCA — Định lý phổ chính là *lý do* PCA hoạt động (Mục 8.11).
- **Ma trận Hessian** trong tối ưu — dấu các trị riêng cho biết cực tiểu / cực đại / yên ngựa.
- **Ma trận Laplace của đồ thị** — trị riêng cho biết cấu trúc cụm; nền của *spectral clustering* và phân tích mạng xã hội.
- A^TA **và** AA^T — luôn đối xứng, nền của SVD (Mục 8.6).
- **Ma trận kernel** trong SVM — đối xứng, nửa xác định dương.

Trong code: với ma trận đối xứng, luôn dùng `numpy.linalg.eigh` thay vì `eig`. Hàm `eigh` khai thác tính đối xứng, chạy nhanh hơn khoảng 2–3 lần, chính xác hơn, và bảo đảm trả về trị riêng *thực* đã sắp xếp cùng ma trận vector riêng *trực giao* — đúng như Định lý 8.2.2 hứa. Dùng `eig` cho ma trận đối xứng sẽ cho bạn phần ảo nhiễu cỡ 10^{-17} và vector riêng không hoàn toàn trực giao.

Ví dụ 8.2.4

Tìm ma trận trực giao P sao cho $P^{-1}AP$ chéo, với $A = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 2 \\ -1 & 2 & 5 \end{bmatrix}$.

Lời giải. Đa thức đặc trưng là $c_A(x) = x(x-1)(x-6)$, nên trị riêng là $\lambda = 0, 1, 6$, với các vector riêng tương ứng

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} -1 \\ 2 \\ 5 \end{bmatrix}$$

Đáng chú ý, các vector riêng này *trực giao* với nhau (không phải may mắn — xem Định lý 8.2.4). Ta có $\|\mathbf{x}_1\|^2 = 6$, $\|\mathbf{x}_2\|^2 = 5$, $\|\mathbf{x}_3\|^2 = 30$, nên

$$P = \begin{bmatrix} \frac{1}{\sqrt{6}}\mathbf{x}_1 & \frac{1}{\sqrt{5}}\mathbf{x}_2 & \frac{1}{\sqrt{30}}\mathbf{x}_3 \end{bmatrix}$$

là ma trận trực giao, và $P^TAP = \text{diag}(0, 1, 6)$.

Định lý 8.2.3

Nếu A là ma trận $n \times n$ *đối xứng*, thì

$$(A\mathbf{x}) \cdot \mathbf{y} = \mathbf{x} \cdot (A\mathbf{y})$$

với mọi cột $\mathbf{x}, \mathbf{y}$ trong $\mathbb{R}^n$.

Chứng minh. Vì $A^T = A$:

$$(A\mathbf{x}) \cdot \mathbf{y} = (A\mathbf{x})^T\mathbf{y} = \mathbf{x}^T A^T\mathbf{y} = \mathbf{x}^T A\mathbf{y} = \mathbf{x} \cdot (A\mathbf{y})$$

■

Định lý 8.2.4

Nếu A đối xứng, thì các vector riêng ứng với các trị riêng *phân biệt* là **trực giao**.

Chứng minh. Cho $A\mathbf{x} = \lambda\mathbf{x}$ và $A\mathbf{y} = \mu\mathbf{y}$ với $\lambda \neq \mu$. Dùng Định lý 8.2.3:

$$\lambda(\mathbf{x} \cdot \mathbf{y}) = (A\mathbf{x}) \cdot \mathbf{y} = \mathbf{x} \cdot (A\mathbf{y}) = \mu(\mathbf{x} \cdot \mathbf{y})$$

Vậy $(\lambda - \mu)(\mathbf{x} \cdot \mathbf{y}) = 0$, nên $\mathbf{x} \cdot \mathbf{y} = 0$. ■

Quy trình chéo hóa trực giao một ma trận đối xứng giờ đã rõ: tìm các trị riêng phân biệt (đều thực), tìm cơ sở *trực chuẩn* cho mỗi không gian riêng (có thể cần Gram–Schmidt). Tập hợp tất cả các vector cơ sở đó là trực chuẩn (theo Định lý 8.2.4) và gồm n vector.

Ví dụ 8.2.5

Chéo hóa trực giao ma trận đối xứng $A = \begin{bmatrix} 8 & -2 & 2 \\ -2 & 5 & 4 \\ 2 & 4 & 5 \end{bmatrix}$.

Lời giải. Đa thức đặc trưng là $c_A(x) = x(x-9)^2$. Vậy các trị riêng phân biệt là 0 và 9, với bội 1 và 2, nên $\dim(E_0) = 1$ và $\dim(E_9) = 2$. Khử Gauss cho

$$E_0(A) = \text{span}\left\{\begin{bmatrix} 1 \\ 2 \\ -2 \end{bmatrix}\right\}, \qquad E_9(A) = \text{span}\left\{\begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}\right\}$$

Hai vector riêng trong E_9 đều trực giao với $\mathbf{x}_1$ (như Định lý 8.2.4 bảo đảm), nhưng *không* trực giao với nhau. Gram–Schmidt cho cơ sở trực giao của $E_9(A)$:

$$\mathbf{x}_2 = \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}, \qquad \mathbf{x}_3 = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix}$$

Chuẩn hóa cho $\{\frac{1}{3}\mathbf{x}_1,\ \frac{1}{\sqrt{5}}\mathbf{x}_2,\ \frac{1}{3\sqrt{5}}\mathbf{x}_3\}$, nên $P = [\ \frac{1}{3}\mathbf{x}_1\ \ \frac{1}{\sqrt{5}}\mathbf{x}_2\ \ \frac{1}{3\sqrt{5}}\mathbf{x}_3\ ]$ là ma trận trực giao với $P^{-1}AP$ chéo.

Đáng chú ý là có những P *tiện hơn*. Chẳng hạn $\mathbf{y}_2 = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$ và $\mathbf{y}_3 = \begin{bmatrix} -2 \\ 2 \\ 1 \end{bmatrix}$ cũng nằm trong $E_9(A)$, trực giao với nhau, và đều có chuẩn 3 (như $\mathbf{x}_1$). Vậy

$$Q = \frac{1}{3}\begin{bmatrix} 1 & 2 & -2 \\ 2 & 1 & 2 \\ -2 & 2 & 1 \end{bmatrix}$$

là một ma trận trực giao đẹp hơn, với $Q^{-1}AQ$ chéo.

Dạng toàn phương và trục chính

Biểu thức $q = ax_1^2 + bx_1x_2 + cx_2^2$ gọi là **dạng toàn phương** (quadratic form) theo các biến x_1, x_2, và đồ thị của $q = 1$ gọi là một *conic*. Chẳng hạn với $q = x_1x_2$, đồ thị $q = 1$ là một hyperbol.

Nếu đặt biến mới $x_1 = y_1 + y_2$, $x_2 = y_1 - y_2$, thì q thành $q = y_1^2 - y_2^2$ — *dạng chéo*, không còn số hạng chéo y_1y_2. Vì vậy các trục y_1, y_2 gọi là **trục chính** của conic. Chéo hóa trực giao cho ta phương pháp hệ thống để tìm trục chính.

Ví dụ 8.2.6

Tìm trục chính của dạng toàn phương $q = x_1^2 - 4x_1x_2 + x_2^2$.
Lời giải. Trước hết viết q dưới dạng ma trận *đối xứng*. Tách số hạng chéo thành hai phần bằng nhau: $q = x_1^2 - 2x_1x_2 - 2x_2x_1 + x_2^2$, nên

$$q = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \mathbf{x}^T A \mathbf{x}$$

Trị riêng của A là $\lambda_1 = 3$, $\lambda_2 = -1$, với vector riêng (trực giao) $\mathbf{x}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ và $\mathbf{x}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Vì $\|\mathbf{x}_1\| = \|\mathbf{x}_2\| = \sqrt{2}$:

$$P = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \text{ trực giao}, \qquad P^T A P = D = \begin{bmatrix} 3 & 0 \\ 0 & -1 \end{bmatrix}$$

Đặt biến mới $\mathbf{y} = P^T\mathbf{x}$, tức

$$y_1 = \tfrac{1}{\sqrt{2}}(x_1 - x_2), \qquad y_2 = \tfrac{1}{\sqrt{2}}(x_1 + x_2)$$

Theo y_1, y_2, dạng q trở thành

$$q = \mathbf{x}^T A \mathbf{x} = (P\mathbf{y})^T A (P\mathbf{y}) = \mathbf{y}^T (P^T A P)\mathbf{y} = 3y_1^2 - y_2^2$$

Chú ý $\mathbf{y} = P^T\mathbf{x}$ thu được từ $\mathbf{x}$ bằng phép quay ngược chiều kim đồng hồ góc $\frac{\pi}{4}$.

Định lý 8.2.5 — Định lý tam giác hóa

Nếu A là ma trận $n \times n$ có n trị riêng *thực*, thì tồn tại ma trận trực giao P sao cho $P^T AP$ là *tam giác trên*.

Chứng minh. Sửa đổi chứng minh Định lý 8.2.2. Nếu $A\mathbf{x}_1 = \lambda_1\mathbf{x}_1$ với $\|\mathbf{x}_1\| = 1$, lấy cơ sở trực chuẩn $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$ của $\mathbb{R}^n$ và $P_1 = [\,\mathbf{x}_1 \ \cdots \ \mathbf{x}_n\,]$. Khi đó $P_1^T A P_1 = \begin{bmatrix} \lambda_1 & B \\ 0 & A_1 \end{bmatrix}$. Theo quy nạp, có Q trực giao với $Q^T A_1 Q = T_1$ tam giác trên. Đặt $P_2 = \begin{bmatrix} 1 & 0 \\ 0 & Q \end{bmatrix}$; khi đó $P = P_1P_2$ trực giao và $P^T AP = \begin{bmatrix} \lambda_1 & BQ \\ 0 & T_1 \end{bmatrix}$ tam giác trên. ■

Hệ quả 8.2.1

Nếu A là ma trận $n \times n$ có các trị riêng thực $\lambda_1, \ldots, \lambda_n$ (có thể trùng), thì

$$\det A = \lambda_1 \lambda_2 \cdots \lambda_n, \qquad \operatorname{tr} A = \lambda_1 + \lambda_2 + \cdots + \lambda_n$$

Hệ quả 8.2.1 là công cụ kiểm tra rất tiện

Hai công thức này đáng thuộc, vì chúng cho bạn cách *kiểm tra nhanh* kết quả tính trị riêng:

- Cộng các trị riêng lại — phải bằng tổng đường chéo của A.
- Nhân các trị riêng — phải bằng $\det A$.

Chẳng hạn ở Ví dụ 8.2.5: trị riêng là $0, 9, 9$. Tổng $= 18 = 8 + 5 + 5$ ✓. Tích $= 0 = \det A$ ✓(ma trận suy biến).
Trong code, sau khi gọi `eigh`, kiểm tra `np.allclose(w.sum(), np.trace(A))` là một assertion rẻ tiền nhưng bắt được rất nhiều lỗi.

8.3 Ma trận xác định dương

Mọi trị riêng của ma trận đối xứng đều thực; mục này bàn về trường hợp các trị riêng đều *dương*. Những ma trận này xuất hiện bất cứ khi nào ta gặp bài toán tối ưu (cực đại, cực tiểu), và có vô số ứng dụng trong khoa học, kỹ thuật, thống kê và hình học.

Định nghĩa 8.5 — Ma trận xác định dương

Ma trận vuông gọi là **xác định dương** (positive definite) nếu nó *đối xứng* và mọi trị riêng λ của nó đều *dương* ($\lambda > 0$).

Định lý 8.3.1

Nếu A xác định dương, thì A khả nghịch và $\det A > 0$.

Chứng minh. $\det A = \lambda_1 \lambda_2 \cdots \lambda_n > 0$ theo Hệ quả 8.2.1. ■

Định lý 8.3.2

Ma trận đối xứng A xác định dương **khi và chỉ khi**

$$\mathbf{x}^T A \mathbf{x} > 0 \qquad \text{với mọi cột } \mathbf{x} \neq \mathbf{0} \text{ trong } \mathbb{R}^n$$

Chứng minh. A đối xứng nên theo Định lý trục chính, $P^T A P = D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ với $P^{-1} = P^T$. Đặt $\mathbf{y} = P^T \mathbf{x}$. Khi đó

$$\mathbf{x}^T A \mathbf{x} = \mathbf{x}^T (P D P^T) \mathbf{x} = \mathbf{y}^T D \mathbf{y} = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2 \tag{8.3}$$

Nếu A xác định dương và $\mathbf{x} \neq \mathbf{0}$, thì $\mathbf{x}^T A \mathbf{x} > 0$ vì có $y_j \neq 0$ nào đó và mọi $\lambda_i > 0$. Ngược lại, nếu $\mathbf{x}^T A \mathbf{x} > 0$ với mọi $\mathbf{x} \neq \mathbf{0}$, lấy $\mathbf{x} = P \mathbf{e}_j$; khi đó $\mathbf{y} = \mathbf{e}_j$ và (8.3) cho $\lambda_j = \mathbf{x}^T A \mathbf{x} > 0$. ■

Ghi chú cho người học — xác định dương là "cái bát ngửa"

Định lý 8.3.2 cho ta cách hiểu *hình học* rất trực quan. Dạng toàn phương $q(\mathbf{x}) = \mathbf{x}^T A\mathbf{x}$ là một mặt cong trong không gian, và:

- A **xác định dương** ($\lambda_i > 0$) $\Rightarrow$ mặt cong là *cái bát ngửa* — có duy nhất một điểm cực tiểu tại gốc.
- A **xác định âm** ($\lambda_i < 0$) $\Rightarrow$ *bát úp* — cực đại tại gốc.
- Trị riêng **trái dấu** $\Rightarrow$ *yên ngựa* — không phải cực trị.

Vì sao tối ưu hóa cần điều này: ma trận Hessian H của hàm mục tiêu tại một điểm dừng quyết định điểm đó là gì. H xác định dương $\Rightarrow$ cực tiểu địa phương. Đây là *tiêu chuẩn đạo hàm bậc hai* bạn học ở giải tích, phát biểu bằng đại số tuyến tính.
Trong machine learning:

- Hàm mất mát *lồi* $\iff$ Hessian nửa xác định dương ở mọi nơi. Đó là lý do hồi quy tuyến tính và logistic regression luôn hội tụ về nghiệm toàn cục, còn mạng neural thì không.
- Ma trận hiệp phương sai luôn nửa xác định dương — và *xác định dương* khi các đặc trưng không có đa cộng tuyến hoàn toàn.
- Ma trận kernel trong SVM phải nửa xác định dương (điều kiện Mercer) — nếu không, bài toán tối ưu không lồi và SVM mất bảo đảm.

Kiểm tra trong code: `np.all(np.linalg.eigvalsh(A) > 0)`, hoặc nhanh hơn: thử `np.linalg.cholesky(A)` — nếu ném `LinAlgError` thì A không xác định dương. Cách thứ hai nhanh gấp đôi và là cách chuẩn trong thực tế.

Ví dụ 8.3.1

Nếu U là ma trận $n \times n$ *khả nghịch* bất kỳ, chứng minh $A = U^T U$ xác định dương.
Lời giải. Với $\mathbf{x} \neq \mathbf{0}$:

$$\mathbf{x}^T A\mathbf{x} = \mathbf{x}^T(U^T U)\mathbf{x} = (U\mathbf{x})^T(U\mathbf{x}) = \|U\mathbf{x}\|^2 > 0$$

vì $U\mathbf{x} \neq \mathbf{0}$ (U khả nghịch). Định lý 8.3.2 áp dụng được.

Ma trận con chính và phân rã Cholesky

Với ma trận $n \times n$ A, ký hiệu ${}^{(r)}A$ là ma trận con $r \times r$ *ở góc trên bên trái* của A. Các ma trận ${}^{(1)}A, {}^{(2)}A, \ldots, {}^{(n)}A = A$ gọi là các **ma trận con chính** (principal submatrices).

Ví dụ 8.3.2

Nếu $A = \begin{bmatrix} 10 & 5 & 2 \\ 5 & 3 & 2 \\ 2 & 2 & 3 \end{bmatrix}$ thì ${}^{(1)}A = [10]$, ${}^{(2)}A = \begin{bmatrix} 10 & 5 \\ 5 & 3 \end{bmatrix}$, và ${}^{(3)}A = A$.

Bổ đề 8.3.1

Nếu A xác định dương, thì mọi ma trận con chính ${}^{(r)}A$ cũng xác định dương.

Chứng minh. Viết $A = \begin{bmatrix} {}^{(r)}A & P \\ Q & R \end{bmatrix}$. Với $\mathbf{y} \neq \mathbf{0}$ trong $\mathbb{R}^r$, đặt $\mathbf{x} = \begin{bmatrix} \mathbf{y} \\ \mathbf{0} \end{bmatrix}$. Khi đó $\mathbf{x} \neq \mathbf{0}$ nên

$$0 < \mathbf{x}^T A \mathbf{x} = \mathbf{y}^T ({}^{(r)}A)\mathbf{y}$$

■

Định lý 8.3.3

Các điều kiện sau tương đương với ma trận đối xứng $n \times n$ A:

1. A xác định dương.
2. $\det({}^{(r)}A) > 0$ với mọi $r = 1, 2, \ldots, n$.
3. $A = U^T U$ với U là ma trận *tam giác trên* có các phần tử đường chéo *dương*.

Hơn nữa, phân rã trong (3) là *duy nhất*, gọi là **phân rã Cholesky** của A.

Ghi chú lịch sử

André-Louis Cholesky (1875–1918) là nhà toán học Pháp, hy sinh trong Thế chiến I. Phân rã của ông được một sĩ quan đồng đội công bố năm 1924, sau khi ông mất.

Thuật toán tìm phân rã Cholesky

Nếu A xác định dương, phân rã Cholesky $A = U^T U$ tìm được như sau:
Bước 1. Đưa A về ma trận tam giác trên U_1 có các phần tử đường chéo dương, chỉ dùng các phép biến đổi hàng dạng *cộng bội của một hàng vào hàng ở dưới*.
Bước 2. Thu U từ U_1 bằng cách *chia mỗi hàng của U_1 cho căn bậc hai của phần tử đường chéo ở hàng đó*.

Ví dụ 8.3.3

Tìm phân rã Cholesky của $A = \begin{bmatrix} 10 & 5 & 2 \\ 5 & 3 & 2 \\ 2 & 2 & 3 \end{bmatrix}$.

Lời giải. A xác định dương theo Định lý 8.3.3 vì $\det {}^{(1)}A = 10 > 0$, $\det {}^{(2)}A = 5 > 0$, $\det {}^{(3)}A = \det A = 3 > 0$.
Bước 1:

$$A = \begin{bmatrix} 10 & 5 & 2 \\ 5 & 3 & 2 \\ 2 & 2 & 3 \end{bmatrix} \to \begin{bmatrix} 10 & 5 & 2 \\ 0 & \frac{1}{2} & 1 \\ 0 & 1 & \frac{13}{5} \end{bmatrix} \to \begin{bmatrix} 10 & 5 & 2 \\ 0 & \frac{1}{2} & 1 \\ 0 & 0 & \frac{3}{5} \end{bmatrix} = U_1$$

Bước 2: chia hàng 1 cho $\sqrt{10}$, hàng 2 cho $\sqrt{1/2}$, hàng 3 cho $\sqrt{3/5}$:

$$U = \begin{bmatrix} \sqrt{10} & \frac{5}{\sqrt{10}} & \frac{2}{\sqrt{10}} \\ 0 & \frac{1}{\sqrt{2}} & \sqrt{2} \\ 0 & 0 & \sqrt{\frac{3}{5}} \end{bmatrix}$$

Bạn đọc có thể kiểm tra $U^T U = A$.

Ghi chú cho người học — Cholesky là "căn bậc hai của ma trận"

Phân rã Cholesky $A = U^T U$ là phiên bản ma trận của việc viết một số dương a thành $a = (\sqrt{a})^2$. Đó là lý do nó được dùng ở khắp nơi:

- **Giải hệ phương trình nhanh gấp đôi.** Với A xác định dương, giải $A\mathbf{x} = \mathbf{b}$ qua Cholesky tốn $\frac{1}{3}n^3$ phép tính, so với $\frac{2}{3}n^3$ của khử Gauss / phân rã LU — nhanh gấp đôi, và ổn định số học mà *không cần xoay hàng* (pivoting).
- **Sinh số ngẫu nhiên có tương quan.** Muốn sinh vector ngẫu nhiên với ma trận hiệp phương sai Σ cho trước: lấy $\mathbf{z}$ gồm các biến chuẩn độc lập, rồi tính $U^T\mathbf{z}$ với $\Sigma = U^T U$. Đây là kỹ thuật chuẩn trong mô phỏng Monte Carlo tài chính và rủi ro.
- **Kiểm tra xác định dương.** Thuật toán Cholesky *thất bại* (gặp căn bậc hai của số âm) chính xác khi A không xác định dương — nên nó là phép kiểm nhanh nhất.
- **Quy trình Gaussian (Gaussian Process)** và **Kalman filter** đều dựa nặng vào Cholesky.

Trong code: `L = numpy.linalg.cholesky(A)`. Chú ý NumPy trả về ma trận *tam giác dưới* L với $A = LL^T$, còn sách này dùng tam giác trên U với $A = U^T U$. Quan hệ: $U = L^T$. Để giải hệ, dùng `scipy.linalg.cho_solve` thay vì tự nhân ngược.

8.4 Phân rã QR

Một trong những ưu điểm chính của ma trận trực giao là dễ nghịch đảo — chuyển vị chính là nghịch đảo. Kết hợp với định lý phân rã trong mục này, ta có cách đơn giản hóa rất nhiều phép tính ma trận (ví dụ trong xấp xỉ bình phương tối thiểu).

Định nghĩa 8.6 — Phân rã QR

Cho A là ma trận $m \times n$ có các cột độc lập. Một **phân rã QR** của A là cách viết

$$A = QR$$

trong đó Q cỡ $m \times n$ có các cột *trực chuẩn*, và R là ma trận khả nghịch *tam giác trên* với các phần tử đường chéo *dương*.

Phân rã này quan trọng vì có những thuật toán máy tính thực hiện được nó với khả năng kiểm soát tốt sai số làm tròn. Về bản chất, đây là *phiên bản ma trận của quá trình Gram–Schmidt*.

Cho $A = [\,\mathbf{c}_1 \;\cdots\; \mathbf{c}_n\,]$ với các cột độc lập. Áp dụng Gram–Schmidt cho các cột này ta được các

cột trực giao $\mathbf{f}_1, \dots, \mathbf{f}_n$. Đặt $\mathbf{q}_k = \frac{1}{\|\mathbf{f}_k\|}\mathbf{f}_k$, ta được các cột trực chuẩn, và

$$\|\mathbf{f}_k\|\mathbf{q}_k = \mathbf{c}_k - (\mathbf{c}_k \cdot \mathbf{q}_1)\mathbf{q}_1 - \cdots - (\mathbf{c}_k \cdot \mathbf{q}_{k-1})\mathbf{q}_{k-1}$$

Biểu diễn ngược lại mỗi $\mathbf{c}_k$ theo các $\mathbf{q}_i$:

$$\begin{aligned}
\mathbf{c}_1 &= \|\mathbf{f}_1\|\mathbf{q}_1 \\
\mathbf{c}_2 &= (\mathbf{c}_2 \cdot \mathbf{q}_1)\mathbf{q}_1 + \|\mathbf{f}_2\|\mathbf{q}_2 \\
\mathbf{c}_3 &= (\mathbf{c}_3 \cdot \mathbf{q}_1)\mathbf{q}_1 + (\mathbf{c}_3 \cdot \mathbf{q}_2)\mathbf{q}_2 + \|\mathbf{f}_3\|\mathbf{q}_3 \\
&\vdots
\end{aligned}$$

Các phương trình này có dạng ma trận chính là phân rã cần tìm:

$$A = [\,\mathbf{q}_1 \ \cdots \ \mathbf{q}_n\,] \begin{bmatrix} \|\mathbf{f}_1\| & \mathbf{c}_2 \cdot \mathbf{q}_1 & \mathbf{c}_3 \cdot \mathbf{q}_1 & \cdots & \mathbf{c}_n \cdot \mathbf{q}_1 \\ 0 & \|\mathbf{f}_2\| & \mathbf{c}_3 \cdot \mathbf{q}_2 & \cdots & \mathbf{c}_n \cdot \mathbf{q}_2 \\ 0 & 0 & \|\mathbf{f}_3\| & \cdots & \mathbf{c}_n \cdot \mathbf{q}_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \|\mathbf{f}_n\| \end{bmatrix} \tag{8.5}$$

Định lý 8.4.1 — Phân rã QR

Mọi ma trận $m \times n$ A có các cột độc lập tuyến tính đều có phân rã QR $A = QR$, trong đó Q có các cột trực chuẩn và R tam giác trên với các phần tử đường chéo dương.

Ví dụ 8.4.1

Tìm phân rã QR của $A = \begin{bmatrix} 1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$.

Lời giải. Áp dụng Gram–Schmidt cho các cột $\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$:

$$\mathbf{f}_1 = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{f}_2 = \mathbf{c}_2 - \tfrac{1}{2}\mathbf{f}_1 = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{f}_3 = \mathbf{c}_3 + \tfrac{1}{2}\mathbf{f}_1 - \mathbf{f}_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Đặt $\mathbf{q}_j = \frac{1}{\|\mathbf{f}_j\|}\mathbf{f}_j$. Khi đó $A = QR$ với

$$Q = \tfrac{1}{\sqrt{6}} \begin{bmatrix} \sqrt{3} & 1 & 0 \\ -\sqrt{3} & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & \sqrt{6} \end{bmatrix}, \qquad R = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 2 & 1 & -1 \\ 0 & \sqrt{3} & \sqrt{3} \\ 0 & 0 & \sqrt{2} \end{bmatrix}$$

Hệ quả 8.4.1

Nếu A có các *hàng* độc lập, thì A phân rã duy nhất thành $A = LP$ với P có các hàng trực chuẩn và L tam giác dưới khả nghịch với đường chéo dương.

Định lý 8.4.2

Mọi ma trận vuông khả nghịch A đều có phân rã $A = QR$ và $A = LP$ với Q, P *trực giao*, R tam giác trên đường chéo dương, L tam giác dưới đường chéo dương.

Định lý 8.4.3 — Tính duy nhất

Cho A là ma trận $m \times n$ có các cột độc lập. Nếu $A = QR$ và $A = Q_1 R_1$ đều là phân rã QR của A, thì $Q_1 = Q$ và $R_1 = R$.

Ghi chú cho người học — QR là cách ĐÚNG để giải bình phương tối thiểu

Ở Mục 5.6 ta giải bình phương tối thiểu bằng hệ phương trình chuẩn tắc $(A^T A)\mathbf{z} = A^T\mathbf{b}$. Mục này cho ta một cách tốt hơn nhiều.
Nếu $A = QR$ thì $Q^T Q = I_n$, nên

$$A^T A = R^T Q^T Q R = R^T R$$

Hệ phương trình chuẩn tắc trở thành $R^T R\mathbf{z} = R^T Q^T\mathbf{b}$, tức

$$\boxed{R\mathbf{z} = Q^T\mathbf{b}}$$

— một hệ *tam giác trên*, giải bằng thế ngược trong $O(n^2)$, không cần nghịch đảo gì cả.
Vì sao cách này tốt hơn nhiều: số điều kiện của $A^T A$ bằng *bình phương* số điều kiện của A. Nếu A đã hơi kém điều kiện ($\kappa(A) = 10^8$), thì $\kappa(A^T A) = 10^{16}$ — vượt quá độ chính xác của số thực 64-bit, và bạn mất *toàn bộ* chữ số có nghĩa. Cách QR làm việc trực tiếp với A, giữ nguyên $\kappa(A)$.

Quy tắc vàng: đừng bao giờ lập ma trận $A^T A$ trong code. Dùng QR hoặc SVD.

Trong code: `Q, R = numpy.linalg.qr(A)` rồi `z = scipy.linalg.solve_triangular(R, Q.T @ b)`. Hoặc đơn giản hơn: `numpy.linalg.lstsq(A, b, rcond=None)` — hàm này dùng SVD, an toàn nhất.
Lưu ý cài đặt: NumPy không dùng Gram–Schmidt để tính QR (kém ổn định), mà dùng *phản xạ Householder*. Gram–Schmidt trong Định lý 8.4.1 là để *chứng minh* phân rã tồn tại, không phải để tính.

8.5 Tính trị riêng bằng máy tính

Trong thực hành, bài toán tìm trị riêng gần như *không bao giờ* được giải bằng cách tìm nghiệm của đa thức đặc trưng. Cách đó rất khó với ma trận lớn; các *phương pháp lặp* tốt hơn nhiều. Mục này mô tả ngắn gọn hai phương pháp.

Phương pháp lũy thừa (Power Method)

Trị riêng λ của A gọi là **trội** (dominant) nếu nó có bội 1 và $|\lambda| > |\mu|$ với mọi trị riêng $\mu \neq \lambda$. Vector riêng tương ứng gọi là *vector riêng trội*.

Khi có trị riêng trội, một kỹ thuật tìm nó là: lấy $\mathbf{x}_0$ làm xấp xỉ đầu, rồi tính

$$\mathbf{x}_{k+1} = A\mathbf{x}_k \qquad \text{với mọi } k \geq 0$$

Kỹ thuật này gọi là **phương pháp lũy thừa** (vì $\mathbf{x}_k = A^k\mathbf{x}_0$). Đồng thời định nghĩa **thương Rayleigh**

$$r_k = \frac{\mathbf{x}_k \cdot \mathbf{x}_{k+1}}{\|\mathbf{x}_k\|^2}, \qquad k \geq 1$$

Khi đó r_k xấp xỉ trị riêng trội λ.

Ví dụ 8.5.1

Dùng phương pháp lũy thừa xấp xỉ vector riêng trội và trị riêng trội của $A = \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}$.

Lời giải. Trị riêng của A là 2 và -1, với vector riêng $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ và $\begin{bmatrix} 1 \\ -2 \end{bmatrix}$. Lấy $\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \ \mathbf{x}_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \ \mathbf{x}_3 = \begin{bmatrix} 5 \\ 6 \end{bmatrix}, \ \mathbf{x}_4 = \begin{bmatrix} 11 \\ 10 \end{bmatrix}, \ \mathbf{x}_5 = \begin{bmatrix} 21 \\ 22 \end{bmatrix}, \ \dots$$

Các vector này tiến dần tới bội vô hướng của vector riêng trội $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Các thương Rayleigh là

$$r_1 = \tfrac{7}{5}, \quad r_2 = \tfrac{27}{13}, \quad r_3 = \tfrac{115}{61}, \quad r_4 = \tfrac{451}{221}, \ \dots$$

tiến dần tới trị riêng trội 2.

Vì sao nó hoạt động. Cho λ_1 trội, $\mathbf{y}_i$ là vector riêng tương ứng, và giả sử $\mathbf{x}_0 = a_1\mathbf{y}_1 + \cdots + a_m\mathbf{y}_m$ với $a_1 \neq 0$. Khi đó

$$\frac{1}{\lambda_1^k}\mathbf{x}_k = a_1\mathbf{y}_1 + a_2\left(\frac{\lambda_2}{\lambda_1}\right)^k\mathbf{y}_2 + \cdots + a_m\left(\frac{\lambda_m}{\lambda_1}\right)^k\mathbf{y}_m$$

Vế phải tiến tới $a_1\mathbf{y}_1$ khi k tăng, vì $|\lambda_i/\lambda_1| < 1$ với mọi $i > 1$.

Ghi chú cho người học — phương pháp lũy thừa CHÍNH LÀ PageRank

Phương pháp lũy thừa trông đơn giản đến mức tầm thường, nhưng nó là thuật toán đã tạo nên Google.

PageRank hoạt động đúng như vậy: xây ma trận chuyển A của đồ thị web (mỗi cột là phân phối xác suất của các liên kết đi ra), rồi lặp $\mathbf{x}_{k+1} = A\mathbf{x}_k$. Vector riêng trội (ứng với $\lambda = 1$) chính là *phân phối dừng* — xác suất một người lướt web ngẫu nhiên đang ở mỗi trang. Đó là điểm PageRank.

Vì sao phương pháp này hợp với dữ liệu lớn:

- Chỉ cần *phép nhân ma trận–vector*, không cần phân rã gì. Với ma trận thưa (sparse) như đồ thị web (mỗi trang chỉ liên kết vài chục trang), mỗi bước tốn $O(\text{số cạnh})$ chứ không phải $O(n^3)$.
- Không cần lưu cả ma trận trong bộ nhớ — chỉ cần tính được $A\mathbf{x}$.
- Dễ song song hóa.

Hạn chế quan trọng:

- Tốc độ hội tụ phụ thuộc tỉ số $|\lambda_2/\lambda_1|$ (gọi là *spectral gap*). Nếu tỉ số gần 1, hội tụ rất chậm. Google giải quyết bằng "damping factor" $0{,}85$ — nó ép spectral gap ra xa.
- Thất bại nếu $\mathbf{x}_0$ không có thành phần theo $\mathbf{y}_1$ ($a_1 = 0$). Trong thực tế chọn $\mathbf{x}_0$ ngẫu nhiên thì xác suất gặp trường hợp này bằng 0.

- Cần chuẩn hóa $\mathbf{x}_k$ sau mỗi bước, nếu không sẽ tràn số (overflow) khi $|\lambda_1| > 1$.

Cùng thuật toán này còn dùng cho: spectral clustering, thuật toán HITS, và bước khởi tạo của nhiều phương pháp SVD xấp xỉ trên dữ liệu lớn.

Thuật toán QR

Một phương pháp tốt hơn nhiều dựa trên phân rã $A = QR$. **Thuật toán QR** dùng nó lặp đi lặp lại để tạo dãy ma trận $A_1 = A, A_2, A_3, \ldots$:

1. Đặt $A_1 = A$ và phân rã $A_1 = Q_1 R_1$.
2. Đặt $A_2 = R_1 Q_1$ (*đảo thứ tự!*) và phân rã $A_2 = Q_2 R_2$.
3. Đặt $A_3 = R_2 Q_2$ và phân rã $A_3 = Q_3 R_3$. ...

Tổng quát: $A_k = Q_k R_k$ và $A_{k+1} = R_k Q_k$. Khi đó A_{k+1} *đồng dạng* với A_k (vì $A_{k+1} = R_k Q_k = Q_k^{-1} A_k Q_k$), nên mọi A_k có cùng trị riêng với A.

Điều đáng kinh ngạc: nếu các trị riêng của A là thực và có *giá trị tuyệt đối khác nhau*, thì dãy $A_1, A_2, A_3, \ldots$ **hội tụ về một ma trận tam giác trên** với các trị riêng nằm trên đường chéo chính.

Ví dụ 8.5.2

Với $A = \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}$, dùng thuật toán QR xấp xỉ các trị riêng.

Lời giải.

$$A_1 = \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix} = Q_1 R_1, \quad Q_1 = \tfrac{1}{\sqrt{5}} \begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix}, \quad R_1 = \tfrac{1}{\sqrt{5}} \begin{bmatrix} 5 & 1 \\ 0 & 2 \end{bmatrix}$$

$$A_2 = R_1 Q_1 = \tfrac{1}{5} \begin{bmatrix} 7 & 9 \\ 4 & -2 \end{bmatrix} = \begin{bmatrix} 1{,}4 & 1{,}8 \\ 0{,}8 & -0{,}4 \end{bmatrix} = Q_2 R_2$$

$$A_3 = R_2 Q_2 = \tfrac{1}{13} \begin{bmatrix} 27 & -5 \\ 8 & -14 \end{bmatrix} = \begin{bmatrix} 2{,}08 & -0{,}38 \\ 0{,}62 & -1{,}08 \end{bmatrix}$$

Dãy này hội tụ về $\begin{bmatrix} 2 & * \\ 0 & -1 \end{bmatrix}$, tức xấp xỉ các trị riêng 2 và -1 trên đường chéo.

Ba cải tiến làm thuật toán QR trở nên thực dụng

Thuật toán QR như mô tả trên hội tụ, nhưng chậm. Ba cải tiến sau biến nó thành thuật toán tìm trị riêng tiêu chuẩn của mọi thư viện tính toán hiện đại:
1. Dịch chuyển (Shifting). Ở bước k, chọn một số s_k và phân rã $A_k - s_k I = Q_k R_k$ thay vì A_k. Khi đó đặt $A_{k+1} = R_k Q_k + s_k I$. Nếu chọn s_k khéo (thường lấy phần tử góc dưới phải), tốc độ hội tụ tăng vọt — từ tuyến tính lên *bậc hai* hoặc *bậc ba*.
2. Chuẩn bị trước (dạng Hessenberg). Ma trận dạng "tam giác trên cộng một đường chéo phụ dưới" gọi là **dạng Hessenberg trên**. Phân rã QR của ma trận như vậy rẻ hơn rất nhiều — $O(n^2)$ thay vì $O(n^3)$. Người ta dùng các *ma trận Householder* $H_1, \ldots, H_m$ trực giao để đưa A về dạng $B = H_m^T \cdots H_1^T A H_1 \cdots H_m$ Hessenberg, rồi chạy thuật toán QR trên B. Vì B đồng dạng với A, trị riêng giữ nguyên.

3. Trị riêng phức. Nếu A thực có trị riêng không thực, thuật toán QR hội tụ về ma trận *tam giác khối*, trong đó các khối chéo hoặc cỡ 1×1 (trị riêng thực) hoặc 2×2 (mỗi khối cho một cặp trị riêng phức liên hợp).
Kết quả: khi bạn gọi `numpy.linalg.eig(A)`, bên trong LAPACK thực hiện đúng ba bước này — Householder đưa về Hessenberg, rồi lặp QR có dịch chuyển. Thuật toán QR được John Francis và Vera Kublanovskaya phát triển độc lập khoảng 1961, và được xếp vào *Top 10 thuật toán của thế kỷ 20*.

8.6 Phân rã giá trị kỳ dị (SVD)

Với ma trận vuông A, việc "chéo hóa" $A = Q^{-1}DQ$ rất hữu dụng — nhưng tiếc là phân rã như vậy *không phải lúc nào cũng tồn tại*. Tuy nhiên, với ma trận thực A *bất kỳ* (vuông hay không), ta luôn tìm được phân rã $A = P\Sigma Q^T$ với P, Q *trực giao* và Σ "chéo". Phân rã đó gọi là **phân rã giá trị kỳ dị** (SVD) — một trong những công cụ hữu dụng nhất của đại số tuyến tính ứng dụng.

Bổ đề 8.6.1

Với mọi ma trận $m \times n$ A: $\operatorname{im} A = \operatorname{col} A$.

8.6.1 Xây dựng SVD

Ta biết rất nhiều về ma trận đối xứng thực: trị riêng thực và chéo hóa trực giao được. Với ma trận thực A bất kỳ, cả A^TA và AA^T đều thực và đối xứng — gợi ý rằng ta học được nhiều về A bằng cách nghiên cứu chúng.

Bổ đề 8.6.2

Cho A là ma trận thực $m \times n$. Khi đó:

1. Trị riêng của A^TA và AA^T đều *thực và không âm*.
2. A^TA và AA^T có *cùng tập trị riêng dương*.

Chứng minh (1). Cả hai ma trận thực và đối xứng nên trị riêng thực. Nếu $A^TA\mathbf{q} = \lambda\mathbf{q}$ với $\mathbf{q} \neq \mathbf{0}$:

$$\|A\mathbf{q}\|^2 = (A\mathbf{q})^T(A\mathbf{q}) = \mathbf{q}^T(A^TA\mathbf{q}) = \lambda\|\mathbf{q}\|^2$$

Vế trái ≥ 0 nên $\lambda \geq 0$.

(2). Nếu $\lambda > 0$ là trị riêng của A^TA với vector riêng $\mathbf{q}$, thì

$$AA^T(A\mathbf{q}) = A[(A^TA)\mathbf{q}] = \lambda(A\mathbf{q})$$

và $A\mathbf{q} \neq \mathbf{0}$. Vậy λ cũng là trị riêng dương của AA^T. Thay A bởi A^T cho chiều ngược lại. ■

Xây dựng. Vì A^TA đối xứng, theo Định lý trục chính lấy cơ sở trực chuẩn $\{\mathbf{q}_1, \ldots, \mathbf{q}_n\}$ của $\mathbb{R}^n$ gồm vector riêng của A^TA, với trị riêng sắp xếp giảm dần:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_r > 0, \qquad \lambda_i = 0 \text{ nếu } i > r$$

Đặt $Q = [\,\mathbf{q}_1 \;\cdots\; \mathbf{q}_n\,]$ — ma trận trực giao. Ta tính

$$A\mathbf{q}_i \cdot A\mathbf{q}_j = \mathbf{q}_i^T(A^TA)\mathbf{q}_j = \lambda_j(\mathbf{q}_i \cdot \mathbf{q}_j)$$

nên $A\mathbf{q}_i \cdot A\mathbf{q}_j = 0$ khi $i \neq j$, và $\|A\mathbf{q}_i\|^2 = \lambda_i$. Suy ra $\{A\mathbf{q}_1, \ldots, A\mathbf{q}_r\}$ là tập trực giao và $A\mathbf{q}_i = \mathbf{0}$ khi $i > r$. Hơn nữa nó là *cơ sở trực giao của* $\operatorname{im} A$, nên

$$\operatorname{rank} A = r$$

Định nghĩa 8.7 — Giá trị kỳ dị

Các số thực $\sigma_i = \sqrt{\lambda_i} = \|A\mathbf{q}_i\|$ với $i = 1, \ldots, n$ gọi là các **giá trị kỳ dị** (singular values) của ma trận A.

Định nghĩa 8.8 — Ma trận kỳ dị

Cho A thực $m \times n$ hạng r, với các giá trị kỳ dị dương $\sigma_1 \geq \cdots \geq \sigma_r > 0$. Định nghĩa

$$D_A = \operatorname{diag}(\sigma_1, \ldots, \sigma_r), \qquad \Sigma_A = \begin{bmatrix} D_A & 0 \\ 0 & 0 \end{bmatrix}_{m \times n}$$

Chuẩn hóa: $\mathbf{p}_i = \frac{1}{\|A\mathbf{q}_i\|} A\mathbf{q}_i$ với $i = 1, \ldots, r$. Khi đó $\{\mathbf{p}_1, \ldots, \mathbf{p}_r\}$ là cơ sở trực chuẩn của $\operatorname{col} A$; dùng Gram–Schmidt mở rộng thành cơ sở trực chuẩn $\{\mathbf{p}_1, \ldots, \mathbf{p}_m\}$ của $\mathbb{R}^m$ và đặt $P = [\,\mathbf{p}_1 \ \cdots \ \mathbf{p}_m\,]$. Vì $\sigma_i \mathbf{p}_i = A\mathbf{q}_i$, ta có $AQ = P\Sigma_A$, tức $A = P\Sigma_A Q^T$.

Định lý 8.6.1

Cho A là ma trận thực $m \times n$ với các giá trị kỳ dị dương $\sigma_1 \geq \cdots \geq \sigma_r > 0$. Khi đó r là *hạng* của A và

$$A = P\Sigma_A Q^T \qquad \text{với } P, Q \text{ trực giao}$$

Ghi chú cho người học — SVD là gì, nói bằng lời

Nếu bạn chỉ nhớ một điều từ cả Chương 8, hãy nhớ điều này:

Mọi ma trận đều là: một phép quay, rồi một phép co giãn theo trục, rồi một phép quay nữa.

$A = P\Sigma Q^T$ đọc từ phải sang trái: Q^T quay, Σ co giãn mỗi trục với hệ số σ_i, P quay tiếp. Không có ma trận nào làm gì phức tạp hơn thế.

Vì sao SVD mạnh hơn chéo hóa:

	Chéo hóa $A = PDP^{-1}$	**SVD** $A = P\Sigma Q^T$
Áp dụng cho	Chỉ ma trận vuông	*Mọi* ma trận $m \times n$
Luôn tồn tại?	Không	**Có, luôn luôn**
P trực giao?	Chỉ khi A đối xứng	**Luôn trực giao**
Các số trên đường chéo	Trị riêng (có thể âm/phức)	Giá trị kỳ dị (luôn ≥ 0 thực)
Ổn định số học	Có thể rất tệ	**Luôn tốt**

Giá trị kỳ dị cho biết gì: σ_i đo "mức độ quan trọng" của hướng thứ i. Cụ thể:

- σ_1 = hệ số giãn lớn nhất = chuẩn $\|A\|_2$ của ma trận.
- Số σ_i khác 0 = **hạng** của A. Đây là cách *duy nhất* đáng tin để tính hạng bằng máy tính (khử Gauss không đáng tin với số thực dấu chấm động).
- $\kappa(A) = \sigma_1/\sigma_r$ = **số điều kiện** — cho biết bài toán nhạy cảm với sai số đến mức nào.
- σ_i nhỏ ≈ 0 báo hiệu ma trận "gần suy biến" — cảnh báo quan trọng khi làm việc với dữ liệu thực.

Ví dụ 8.6.1

Tìm một SVD cho $A = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 1 & 0 \end{bmatrix}$.

Lời giải. Ta có $A^TA = \begin{bmatrix} 2 & -1 & 1 \\ -1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$, với đa thức đặc trưng $(x-3)(x-1)x$. Vậy trị riêng (giảm dần) là $\lambda_1 = 3$, $\lambda_2 = 1$, $\lambda_3 = 0$, với các vector riêng đơn vị

$$\mathbf{q}_1 = \tfrac{1}{\sqrt{6}}\begin{bmatrix} 2 \\ -1 \\ 1 \end{bmatrix}, \quad \mathbf{q}_2 = \tfrac{1}{\sqrt{2}}\begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{q}_3 = \tfrac{1}{\sqrt{3}}\begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

Giá trị kỳ dị là $\sigma_1 = \sqrt{3}$, $\sigma_2 = 1$, $\sigma_3 = 0$, nên $\operatorname{rank}(A) = 2$ và

$$\Sigma_A = \begin{bmatrix} \sqrt{3} & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Tính $A\mathbf{q}_1 = \frac{\sqrt{6}}{2}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$, $A\mathbf{q}_2 = \frac{\sqrt{2}}{2}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $A\mathbf{q}_3 = \mathbf{0}$. Chuẩn hóa cho

$$\mathbf{p}_1 = \tfrac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}, \qquad \mathbf{p}_2 = \tfrac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Vậy $P = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$ và $A = P\Sigma_A Q^T$.

Thuật toán tìm SVD

Cho ma trận thực $m \times n$ A, tìm SVD $A = P\Sigma_A Q^T$ như sau:

1. Tìm các trị riêng (thực, không âm) $\lambda_1, \ldots, \lambda_n$ của A^TA với vector riêng *trực chuẩn* $\mathbf{q}_1, \ldots, \mathbf{q}_n$. Sắp xếp sao cho $\lambda_1 \geq \cdots \geq \lambda_r > 0$ và $\lambda_i = 0$ khi $i > r$.
2. Số nguyên r chính là hạng của A.
3. $Q = [\,\mathbf{q}_1 \ \cdots \ \mathbf{q}_n\,]$.

4. Đặt $\mathbf{p}_i = \frac{1}{\|A\mathbf{q}_i\|} A\mathbf{q}_i$ với $i \le r$, rồi mở rộng thành cơ sở trực chuẩn của $\mathbb{R}^m$ (bằng Gram–Schmidt).
5. $P = [\,\mathbf{p}_1 \ \cdots \ \mathbf{p}_m\,]$.
6. $\sigma_i = \sqrt{\lambda_i}$, và $\Sigma_A = \begin{bmatrix} \operatorname{diag}(\sigma_1, \ldots, \sigma_r) & 0 \\ 0 & 0 \end{bmatrix}_{m \times n}$.
7. $A = P \Sigma_A Q^T$.

Cảnh báo thực hành

Thuật toán trên là để *chứng minh SVD tồn tại* và để tính tay bài tập nhỏ. Trong thực tế, **không ai** tính SVD bằng cách lập $A^T A$ — vì như đã nói ở Mục 8.4, việc đó bình phương số điều kiện và làm mất một nửa số chữ số có nghĩa.
Các thư viện dùng thuật toán Golub–Kahan: đưa A về dạng *song chéo* (bidiagonal) bằng phản xạ Householder, rồi chạy một biến thể của thuật toán QR. Trong code: `U, s, Vt = numpy.linalg.svd(A)` — chú ý NumPy trả về `s` là *mảng* các giá trị kỳ dị, không phải ma trận, và trả về V^T chứ không phải V.

8.6.2 Bốn không gian cơ bản

Định nghĩa 8.10 — Bốn không gian cơ bản

Các **không gian cơ bản** (fundamental subspaces) của ma trận $m \times n$ A là:

$$\operatorname{row} A, \qquad \operatorname{col} A, \qquad \operatorname{null} A, \qquad \operatorname{null} A^T$$

Bổ đề 8.6.4

Với ma trận A bất kỳ:

1. $(\operatorname{row} A)^\perp = \operatorname{null} A$ và $(\operatorname{col} A)^\perp = \operatorname{null} A^T$.
2. Nếu U là không gian con của $\mathbb{R}^n$ thì $U^{\perp\perp} = U$.
3. Nếu $\{\mathbf{f}_1, \ldots, \mathbf{f}_m\}$ là cơ sở trực chuẩn của $\mathbb{R}^m$ và $U = \operatorname{span}\{\mathbf{f}_1, \ldots, \mathbf{f}_k\}$, thì $U^\perp = \operatorname{span}\{\mathbf{f}_{k+1}, \ldots, \mathbf{f}_m\}$.

Định lý 8.6.2

Cho $A = P\Sigma Q^T$ là SVD bất kỳ của A, với $P = [\,\mathbf{p}_1 \ \cdots \ \mathbf{p}_m\,]$ và $Q = [\,\mathbf{q}_1 \ \cdots \ \mathbf{q}_n\,]$. Khi đó:

1. $r = \operatorname{rank} A$, và các giá trị kỳ dị của A là $d_1, \ldots, d_r$.
2. Bốn không gian cơ bản được mô tả bởi:
   a. $\{\mathbf{p}_1, \ldots, \mathbf{p}_r\}$ là cơ sở trực chuẩn của $\operatorname{col} A$;
   b. $\{\mathbf{p}_{r+1}, \ldots, \mathbf{p}_m\}$ là cơ sở trực chuẩn của $\operatorname{null} A^T$;
   c. $\{\mathbf{q}_{r+1}, \ldots, \mathbf{q}_n\}$ là cơ sở trực chuẩn của $\operatorname{null} A$;
   d. $\{\mathbf{q}_1, \ldots, \mathbf{q}_r\}$ là cơ sở trực chuẩn của $\operatorname{row} A$.

Ghi chú cho người học — ”bức tranh lớn” của đại số tuyến tính

Định lý 8.6.2 là kết quả mà Gilbert Strang gọi là *Bức tranh lớn của Đại số tuyến tính*. Nó nói: *một lần chạy SVD cho bạn cơ sở trực chuẩn của cả bốn không gian cơ bản, miễn phí.*
Cấu trúc:

$$\mathbb{R}^n = \underbrace{\operatorname{row} A}_{\dim=r} \oplus \underbrace{\operatorname{null} A}_{\dim=n-r}, \qquad \mathbb{R}^m = \underbrace{\operatorname{col} A}_{\dim=r} \oplus \underbrace{\operatorname{null} A^T}_{\dim=m-r}$$

và A ánh xạ $\operatorname{row} A$ *song ánh* lên $\operatorname{col} A$, còn nghiền $\operatorname{null} A$ về $\mathbf{0}$.
Ý nghĩa thực tế: khi giải $A\mathbf{x} = \mathbf{b}$:

- $\mathbf{b} \in \operatorname{col} A$? Nếu không, hệ vô nghiệm $\to$ dùng bình phương tối thiểu.
- $\operatorname{null} A \neq \{\mathbf{0}\}$? Nếu có, nghiệm không duy nhất $\to$ có vô số nghiệm, khác nhau bởi một phần tử của $\operatorname{null} A$.
- Nghiệm có *chuẩn nhỏ nhất* luôn nằm trong $\operatorname{row} A$ — đó chính là nghiệm mà nghịch đảo giả A^+ trả về.

8.6.3 Phân rã cực của ma trận vuông thực

Định nghĩa 8.11 — Ma trận nửa xác định dương

Ma trận thực $n \times n$ G gọi là **dương** (positive, hay *nửa xác định dương*) nếu nó đối xứng và $\mathbf{x}^T G\mathbf{x} \geq 0$ với mọi $\mathbf{x} \in \mathbb{R}^n$.

Định nghĩa 8.12 — Phân rã cực

Nếu A là ma trận thực $n \times n$, phân rã

$$A = GQ \qquad (G \text{ dương},\ Q \text{ trực giao})$$

gọi là một **phân rã cực** (polar decomposition) của A.

Định lý 8.6.3

Mọi ma trận vuông thực đều có phân rã cực.

Chứng minh. Cho $A = U\Sigma V^T$ là SVD của A. Vì $U^TU = I_n$:

$$A = U\Sigma V^T = (U\Sigma)(U^TU)V^T = (U\Sigma U^T)(UV^T)$$

Đặt $G = U\Sigma U^T$ và $Q = UV^T$; khi đó Q trực giao và G dương. ■

Vì sao gọi là ”phân rã cực”

Tên gọi gợi nhớ *dạng cực* của số phức $z = re^{i\theta}$, và đó không phải trùng hợp.
Đồng nhất số phức $a + bi$ với ma trận $\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$. Khi đó

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix} = \begin{bmatrix} r & 0 \\ 0 & r \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} = GQ$$

với G dương (tương ứng $r = |z|$) và Q trực giao (tương ứng $e^{i\theta}$). Đây chính là $z = re^{i\theta}$ viết

dưới dạng ma trận. Định nghĩa 8.12 chỉ đơn giản mở rộng thuật ngữ cho ma trận $n \times n$.
Ứng dụng trong đồ họa và robotics: phân rã cực tách một biến đổi bất kỳ thành "phần biến dạng" (G) và "phần quay thuần túy" (Q). Điều này dùng để:

- *Trực giao hóa lại* ma trận quay sau nhiều phép nhân tích lũy sai số — lấy Q từ phân rã cực là ma trận quay gần nhất.
- Nội suy giữa hai tư thế (pose) trong animation — nội suy Q và G riêng cho kết quả tự nhiên hơn nội suy trực tiếp ma trận.
- Trong cơ học môi trường liên tục, G chính là *tensor biến dạng* và Q là chuyển động cứng.

8.6.4 Nghịch đảo giả của ma trận

Ma trận không vuông không thể có nghịch đảo. Tuy nhiên có một ứng viên thay thế.

Định lý 8.6.4 — Định lý Penrose

Cho ma trận thực $m \times n$ A bất kỳ, tồn tại *đúng một* ma trận $n \times m$ B thỏa:

P1 $ABA = A$ và $BAB = B$;

P2 cả AB và BA đều *đối xứng*.

Định nghĩa 8.13 — Nghịch đảo giả

Nghịch đảo giả (pseudoinverse) của A là ma trận duy nhất A^+ thỏa P1 và P2. Nó còn được gọi là *nghịch đảo Moore–Penrose*.

Nếu A khả nghịch thì $A^+ = A^{-1}$. Nói chung $A^{++} = A$.

Định lý 8.6.5

Cho A là ma trận $m \times n$.

1. Nếu $\operatorname{rank} A = m$ thì AA^T khả nghịch và $A^+ = A^T(AA^T)^{-1}$.
2. Nếu $\operatorname{rank} A = n$ thì A^TA khả nghịch và $A^+ = (A^TA)^{-1}A^T$.

Định nghĩa 8.14

Nếu $\Sigma = \begin{bmatrix} D & 0 \\ 0 & 0 \end{bmatrix}_{m \times n}$ với $D = \operatorname{diag}(d_1, \ldots, d_r)$, $d_i > 0$, đặt

$$\Sigma' = \begin{bmatrix} D^{-1} & 0 \\ 0 & 0 \end{bmatrix}_{n \times m}$$

Định lý 8.6.6

Cho A thực $m \times n$ và $A = P\Sigma Q^T$ là SVD bất kỳ của A. Khi đó

$$A^+ = Q\Sigma' P^T$$

Ví dụ 8.6.3

Tìm A^+ nếu $A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$.

Lời giải. $A^TA = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ với trị riêng $\lambda_1 = 1$, $\lambda_2 = 0$ và vector riêng $\mathbf{e}_1, \mathbf{e}_2$, nên $Q = I_2$.

A có hạng 1 với $\sigma_1 = 1$, $\sigma_2 = 0$, nên $\Sigma_A = A$ và $\Sigma'_A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$. Tính $\mathbf{p}_1 = \mathbf{e}_1$, mở rộng thành cơ sở chuẩn tắc, nên $P = I_3$. Vậy

$$A^+ = Q\Sigma'_A P^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = A^T$$

Bổ đề 8.6.7 — Tính chất của nghịch đảo giả

Cho A là ma trận $m \times n$.

1. $A^{++} = A$.
2. Nếu A khả nghịch thì $A^+ = A^{-1}$.
3. $(A^T)^+ = (A^+)^T$.
4. $(kA)^+ = k^{-1}A^+$ với mọi $k \neq 0$.
5. $(PAQ)^+ = P^T(A^+)Q^T$ với P, Q trực giao.

Ghi chú cho người học — SVD trong công việc thực tế

Đây là phần quan trọng nhất của cả chương với ngành của bạn. SVD làm được bốn việc lớn:

1. Xấp xỉ hạng thấp (nén dữ liệu). Viết

$$A = \sigma_1\mathbf{p}_1\mathbf{q}_1^T + \sigma_2\mathbf{p}_2\mathbf{q}_2^T + \cdots + \sigma_r\mathbf{p}_r\mathbf{q}_r^T$$

Giữ lại k số hạng đầu cho *xấp xỉ tốt nhất có thể* bằng ma trận hạng k (định lý Eckart–Young). Với ảnh 1000×1000: lưu đủ cần 10^6 số; giữ $k = 50$ chỉ cần $50\times(1000+1000+1) \approx 10^5$ số — nén 10 lần mà mắt thường khó phân biệt.

2. PCA. Thành phần chính chính là các $\mathbf{q}_i$ của dữ liệu đã quy tâm; phương sai giải thích bởi thành phần i tỉ lệ với σ_i^2. (Chi tiết ở Mục 8.11.)

3. Hệ gợi ý. Ma trận người dùng $\times$ sản phẩm rất thưa; SVD hạng thấp "điền vào" các ô trống — đây là ý tưởng cốt lõi của *collaborative filtering*. Netflix Prize được giải bằng biến thể của kỹ thuật này.

4. Nghịch đảo giả và bình phương tối thiểu. $A^+\mathbf{b}$ luôn cho nghiệm tốt nhất, kể cả khi A suy biến hoặc không vuông:

- Hệ vô nghiệm $\to A^+\mathbf{b}$ là nghiệm bình phương tối thiểu.
- Hệ vô số nghiệm $\to A^+\mathbf{b}$ là nghiệm có *chuẩn nhỏ nhất*.

Mẹo quan trọng: `numpy.linalg.pinv` có tham số `rcond` — nó *bỏ qua* các σ_i nhỏ hơn ngưỡng. Đây gọi là *truncated SVD* hoặc *regularization*, và nó cực kỳ quan trọng: các σ_i

rất nhỏ thường chỉ là nhiễu, và chia cho chúng ($1/\sigma_i$ rất lớn) sẽ khuếch đại nhiễu thành vô nghĩa. Cắt bỏ chúng cho kết quả ổn định hơn nhiều.
Chi phí: SVD đầy đủ tốn $O(mn\min(m,n))$ — khá đắt với ma trận lớn. Với dữ liệu lớn, dùng `scipy.sparse.linalg.svds` (chỉ tính k giá trị lớn nhất) hoặc `sklearn.decomposition.TruncatedSVD` (dùng thuật toán ngẫu nhiên hóa, nhanh hơn rất nhiều).

8.7 Ma trận phức

Trị riêng của ma trận thực có thể là số phức. Thực tế, gần như mọi thứ trong sách này vẫn đúng nếu thay "số thực" bằng "số phức" ở mọi nơi. Mục này phát triển lý thuyết đủ xa để chứng minh lại rằng trị riêng của ma trận đối xứng thực là thực, và để chứng minh *định lý phổ*.

Ký hiệu $\mathbb{C}^n$ là tập mọi bộ n số phức, với phép cộng và nhân vô hướng (phức) định nghĩa như thường lệ. $\mathbb{C}^n$ là không gian vector với vô hướng phức.

Với ma trận phức $A = [a_{ij}]$, **ma trận liên hợp** là $\overline{A} = [\overline{a_{ij}}]$. Ta có $\overline{A+B} = \overline{A} + \overline{B}$ và $\overline{AB} = \overline{A}\,\overline{B}$.

Tích trong chuẩn tắc

Định nghĩa 8.15 — Tích trong chuẩn tắc trong $\mathbb{C}^n$

Cho $\mathbf{z} = (z_1, \ldots, z_n)$ và $\mathbf{w} = (w_1, \ldots, w_n)$ trong $\mathbb{C}^n$, định nghĩa **tích trong chuẩn tắc**

$$\langle \mathbf{z}, \mathbf{w} \rangle = z_1\overline{w_1} + z_2\overline{w_2} + \cdots + z_n\overline{w_n}$$

Ví dụ 8.7.1

Nếu $\mathbf{z} = (2,\ 1-i,\ 2i,\ 3-i)$ và $\mathbf{w} = (1-i,\ -1,\ -i,\ 3+2i)$, thì

$$\langle \mathbf{z}, \mathbf{w} \rangle = 2(1+i) + (1-i)(-1) + (2i)(i) + (3-i)(3-2i) = 6 - 6i$$

$$\langle \mathbf{z}, \mathbf{z} \rangle = |2|^2 + |1-i|^2 + |2i|^2 + |3-i|^2 = 4 + 2 + 4 + 10 = 20$$

Định lý 8.7.1

Cho $\mathbf{z}, \mathbf{z}_1, \mathbf{w}, \mathbf{w}_1$ trong $\mathbb{C}^n$ và $\lambda \in \mathbb{C}$.

1. $\langle \mathbf{z} + \mathbf{z}_1, \mathbf{w} \rangle = \langle \mathbf{z}, \mathbf{w} \rangle + \langle \mathbf{z}_1, \mathbf{w} \rangle$ và tương tự ở biến thứ hai.
2. $\langle \lambda\mathbf{z}, \mathbf{w} \rangle = \lambda\langle \mathbf{z}, \mathbf{w} \rangle$ và $\langle \mathbf{z}, \lambda\mathbf{w} \rangle = \overline{\lambda}\langle \mathbf{z}, \mathbf{w} \rangle$.
3. $\langle \mathbf{z}, \mathbf{w} \rangle = \overline{\langle \mathbf{w}, \mathbf{z} \rangle}$.
4. $\langle \mathbf{z}, \mathbf{z} \rangle \geq 0$, và $= 0$ khi và chỉ khi $\mathbf{z} = \mathbf{0}$.

Vì sao phải lấy liên hợp trong định nghĩa tích trong

Nếu định nghĩa $\langle \mathbf{z}, \mathbf{w} \rangle = \sum z_i w_i$ (không liên hợp), thì với $\mathbf{z} = (1, i)$ ta được $\langle \mathbf{z}, \mathbf{z} \rangle = 1 + i^2 = 0$ dù $\mathbf{z} \neq \mathbf{0}$! Khái niệm "độ dài" sẽ sụp đổ.
Liên hợp sửa điều đó: $\langle \mathbf{z}, \mathbf{z} \rangle = \sum |z_i|^2$ luôn là số thực không âm, và bằng 0 chỉ khi $\mathbf{z} = \mathbf{0}$.

Giá phải trả là tính chất (3): tích trong *không còn đối xứng* mà chỉ đối xứng-liên hợp, và (2): nó *tuyến tính liên hợp* ở biến thứ hai.
Đây là quy ước cơ bản trong xử lý tín hiệu và cơ học lượng tử. (Chú ý: vật lý thường dùng quy ước ngược lại — liên hợp ở biến *thứ nhất*.)

Định nghĩa 8.16 — Chuẩn trong $\mathbb{C}^n$

$$\|\mathbf{z}\| = \sqrt{\langle \mathbf{z}, \mathbf{z} \rangle} = \sqrt{|z_1|^2 + \cdots + |z_n|^2}$$

Định nghĩa 8.17 — Chuyển vị liên hợp

Chuyển vị liên hợp A^H của ma trận phức A được định nghĩa bởi

$$A^H = (\overline{A})^T = \overline{(A^T)}$$

Chú ý $A^H = A^T$ khi A thực.

Định lý 8.7.3

Với ma trận phức A, B và $\lambda \in \mathbb{C}$:

$$(A^H)^H = A, \quad (A+B)^H = A^H + B^H, \quad (\lambda A)^H = \overline{\lambda} A^H, \quad (AB)^H = B^H A^H$$

Ma trận Hermite và ma trận unita

Định nghĩa 8.18 — Ma trận Hermite

Ma trận phức vuông A gọi là **Hermite** (hermitian) nếu $A^H = A$.

Ma trận Hermite dễ nhận ra: các phần tử trên đường chéo chính phải *thực*, và phần tử đối xứng qua đường chéo phải là *liên hợp* của nhau.

Ví dụ 8.7.4

$\begin{bmatrix} 3 & i & 2+i \\ -i & -2 & -7 \\ 2-i & -7 & 1 \end{bmatrix}$ là Hermite; còn $\begin{bmatrix} 1 & i \\ i & -2 \end{bmatrix}$ và $\begin{bmatrix} 1 & i \\ -i & i \end{bmatrix}$ thì không.

Định lý 8.7.4

Ma trận phức $n \times n$ A là Hermite **khi và chỉ khi**

$$\langle A\mathbf{z}, \mathbf{w} \rangle = \langle \mathbf{z}, A\mathbf{w} \rangle$$

với mọi $\mathbf{z}, \mathbf{w}$ trong $\mathbb{C}^n$.

Định lý 8.7.5

Cho A là ma trận Hermite.

1. Các trị riêng của A đều **thực**.
2. Vector riêng ứng với các trị riêng phân biệt là **trực giao**.

Chứng minh. Cho $A\mathbf{z} = \lambda\mathbf{z}$ và $A\mathbf{w} = \mu\mathbf{w}$. Định lý 8.7.4 cho

$$\lambda\langle \mathbf{z}, \mathbf{w}\rangle = \langle A\mathbf{z}, \mathbf{w}\rangle = \langle \mathbf{z}, A\mathbf{w}\rangle = \overline{\mu}\langle \mathbf{z}, \mathbf{w}\rangle \tag{8.6}$$

Nếu $\mu = \lambda$ và $\mathbf{w} = \mathbf{z}$, ta được $\lambda\langle \mathbf{z}, \mathbf{z}\rangle = \overline{\lambda}\langle \mathbf{z}, \mathbf{z}\rangle$. Vì $\langle \mathbf{z}, \mathbf{z}\rangle = \|\mathbf{z}\|^2 \neq 0$, suy ra $\lambda = \overline{\lambda}$, tức λ thực. Tương tự μ thực, nên (8.6) cho $\lambda\langle \mathbf{z}, \mathbf{w}\rangle = \mu\langle \mathbf{z}, \mathbf{w}\rangle$. Nếu $\lambda \neq \mu$ thì $\langle \mathbf{z}, \mathbf{w}\rangle = 0$. ■

Định nghĩa 8.20 — Ma trận unita

Ma trận phức vuông U gọi là **unita** (unitary) nếu $U^{-1} = U^{H}$.

Vậy ma trận thực là unita khi và chỉ khi nó trực giao.

Định lý 8.7.6

Các điều kiện sau tương đương với ma trận phức $n \times n$ A:

1. A khả nghịch và $A^{-1} = A^{H}$;
2. các *hàng* của A trực chuẩn trong $\mathbb{C}^n$;
3. các *cột* của A trực chuẩn trong $\mathbb{C}^n$.

Ví dụ 8.7.6

Xét ma trận Hermite $A = \begin{bmatrix} 3 & 2+i \\ 2-i & 7 \end{bmatrix}$. Tìm trị riêng, hai vector riêng trực chuẩn, và ma trận unita U sao cho $U^{H}AU$ chéo.

Lời giải. Đa thức đặc trưng là

$$c_A(x) = \det \begin{bmatrix} x-3 & -2-i \\ -2+i & x-7 \end{bmatrix} = (x-2)(x-8)$$

Trị riêng là 2 và 8 (đều thực, đúng như dự đoán), với vector riêng tương ứng $\begin{bmatrix} 2+i \\ -1 \end{bmatrix}$ và $\begin{bmatrix} 1 \\ 2-i \end{bmatrix}$ (trực giao, đúng như dự đoán). Mỗi vector có độ dài $\sqrt{6}$, nên

$$U = \tfrac{1}{\sqrt{6}} \begin{bmatrix} 2+i & 1 \\ -1 & 2-i \end{bmatrix}$$

là ma trận unita, và $U^{H}AU = \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}$.

Định lý Schur và Định lý phổ

Định lý 8.7.7 — Định lý Schur

Nếu A là ma trận phức $n \times n$ bất kỳ, tồn tại ma trận *unita* U sao cho $U^H AU = T$ là **tam giác trên**. Hơn nữa, các phần tử đường chéo của T chính là các trị riêng của A (kể cả bội).

Hệ quả 8.7.1

Nếu A là ma trận phức $n \times n$ với các trị riêng $\lambda_1, \ldots, \lambda_n$ (kể cả bội), thì

$$\det A = \lambda_1 \lambda_2 \cdots \lambda_n, \qquad \operatorname{tr} A = \lambda_1 + \cdots + \lambda_n$$

Định lý 8.7.8 — Định lý phổ (Spectral Theorem)

Nếu A là ma trận *Hermite*, thì tồn tại ma trận *unita* U sao cho $U^H AU$ là ma trận **chéo** (với các phần tử chéo thực).

Chứng minh. Theo Định lý Schur, lấy $U^H AU = T$ tam giác trên với U unita. Vì A Hermite, $T^H = (U^H AU)^H = U^H A^H U = U^H AU = T$, nên T vừa tam giác trên vừa Hermite — do đó T phải chéo. ■

Ma trận chuẩn tắc

Ma trận phức $n \times n$ N gọi là **chuẩn tắc** (normal) nếu $NN^H = N^H N$.

Mọi ma trận Hermite và mọi ma trận unita đều chuẩn tắc.

Định lý 8.7.9

Ma trận phức $n \times n$ A **chéo hóa unita được** khi và chỉ khi A **chuẩn tắc**.

Định lý 8.7.10 — Định lý Cayley–Hamilton

Nếu A là ma trận vuông với đa thức đặc trưng $c_A(x)$, thì

$$c_A(A) = 0$$

Ghi chú cho người học — vì sao ma trận phức quan trọng trong kỹ thuật

Mục này nhìn có vẻ trừu tượng, nhưng ma trận phức là ngôn ngữ của *mọi thứ dao động hoặc tuần hoàn*.

1. Xử lý tín hiệu và biến đổi Fourier. Ma trận DFT (Discrete Fourier Transform) cỡ $n \times n$ có phần tử $F_{jk} = \frac{1}{\sqrt{n}} e^{-2\pi ijk/n}$. Ma trận này là **unita**! Đó là lý do biến đổi Fourier bảo toàn năng lượng (định lý Parseval), và tại sao biến đổi ngược chỉ đơn giản là F^H. Thuật toán FFT tính $F\mathbf{x}$ trong $O(n \log n)$ thay vì $O(n^2)$.

2. Cơ học lượng tử. Trạng thái là vector đơn vị trong $\mathbb{C}^n$; đại lượng quan sát được là ma trận *Hermite* (nên trị riêng — kết quả đo — luôn thực, Định lý 8.7.5); tiến hóa theo thời gian là ma trận *unita* (nên xác suất tổng luôn bằng 1). Máy tính lượng tử: mỗi cổng lượng tử là một ma trận unita $2^k \times 2^k$.

3. Hệ điều khiển và độ ổn định. Trị riêng phức $\lambda = p + iq$ của ma trận hệ thống: p

quyết định ổn định (tắt dần nếu $p < 0$), q quyết định tần số dao động. Đúng như Mục 6.6.
4. Định lý Cayley–Hamilton có ứng dụng thực tế: nó cho thấy A^n biểu diễn được qua $I, A, \ldots, A^{n-1}$. Nhờ đó tính A^k với k lớn chỉ cần các lũy thừa thấp — kỹ thuật dùng trong lý thuyết điều khiển và một số thuật toán mật mã.
Trong code: NumPy hỗ trợ đầy đủ ma trận phức. `A.conj().T` cho A^H (hoặc `A.conj().T` viết gọn là `A.H` trong `numpy.matrix`, nhưng lớp đó đã lỗi thời). Với ma trận Hermite, dùng `numpy.linalg.eigh` — nó xử lý cả trường hợp phức.

8.8 Ứng dụng: Mã tuyến tính trên trường hữu hạn

Từ nhiều thế kỷ, con người đã dùng mã để truyền thông điệp. Trong nhiều trường hợp — truyền thông tin tài chính, y tế, quân sự — thông điệp được che giấu để kẻ chặn bắt không hiểu được; chủ đề đó gọi là *mật mã học* (cryptography). Ở đây ta nghiên cứu một vấn đề khác: **phát hiện và sửa lỗi** trong quá trình truyền.

Những bức ảnh tuyệt đẹp về Sao Thổ do tàu thăm dò gửi về là ví dụ rất tốt về mức độ thành công của các phương pháp này. Tín hiệu chịu *nhiễu* (như nhiễu mặt trời) gây lỗi, và các lỗi đó phải được phát hiện và sửa trước khi in được ảnh chất lượng cao. Việc đó dùng **mã sửa lỗi** (error-correcting codes).

Số học modulo

Với số nguyên $n \geq 2$, mọi số nguyên a viết được duy nhất

$$a = qn + r \qquad \text{với } 0 \leq r \leq n-1$$

Ở đây q là *thương*, r là *số dư* của a modulo n. Tập các số dư

$$\mathbb{Z}_n = \{0, 1, 2, \ldots, n-1\}$$

gọi là **tập số nguyên modulo** n. Ta cộng và nhân trong $\mathbb{Z}_n$ bằng cách tính như bình thường rồi lấy số dư modulo n.

Ví dụ trong $\mathbb{Z}_6$: $2+5=1$, $2\cdot 5=4$, $3+5=2$, $3+3=0$. Các luật số học thông thường vẫn đúng, nhưng có khác biệt: $3 \cdot 4 = 0$ trong $\mathbb{Z}_6$ dù $3 \neq 0$ và $4 \neq 0$.

Nói chung: $a = b$ trong $\mathbb{Z}_n$ khi và chỉ khi $a - b$ là bội của n; ta viết $a \equiv b \pmod{n}$.

Trường hữu hạn

Một hệ đóng với cộng và nhân, thỏa các luật số học thông thường, và trong đó *mọi phần tử khác không đều có nghịch đảo*, gọi là một **trường** (field). $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Q}$ đều là trường; $\mathbb{Z}$ thì *không* (vì 2 không có nghịch đảo nguyên).

Điều quan trọng: *gần như mọi thứ ta đã làm vẫn đúng nếu vô hướng lấy từ một trường bất kỳ*. Khử Gauss vẫn giải được hệ; ma trận vuông khả nghịch khi và chỉ khi định thức khác 0; thuật toán tìm nghịch đảo vẫn hoạt động. Lý do: các chứng minh chỉ dùng các tính chất của trường.

Số nguyên $p \geq 2$ gọi là **số nguyên tố** nếu nó không phân tích được thành tích hai số nguyên dương khác 1.

Định lý 8.8.1

Nếu p là số nguyên tố, thì $\mathbb{Z}_p$ là một **trường** với phép cộng và nhân modulo p.

Ngược lại, nếu n không nguyên tố, $n = ab$ với $2 \le a, b \le n-1$, thì $ab = 0$ trong $\mathbb{Z}_n$ nên b không có nghịch đảo — $\mathbb{Z}_n$ không là trường.

Ví dụ 8.8.3

Xác định xem $A = \begin{bmatrix} 1 & 4 \\ 6 & 5 \end{bmatrix}$ trên $\mathbb{Z}_7$ có khả nghịch không, và tìm nghịch đảo.

Lời giải. Trong $\mathbb{Z}_7$: $\det A = 1 \cdot 5 - 6 \cdot 4 = 5 - 24 = 5 - 3 = 2 \neq 0$, nên A khả nghịch. Ta có $A^{-1} = 2^{-1} \begin{bmatrix} 5 & -4 \\ -6 & 1 \end{bmatrix}$. Chú ý $2^{-1} = 4$ trong $\mathbb{Z}_7$ (vì $2 \cdot 4 = 8 = 1$), và $-4 = 3$, $-6 = 1$. Vậy

$$A^{-1} = 4 \begin{bmatrix} 5 & 3 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 5 \\ 4 & 4 \end{bmatrix}$$

Ngoài các $\mathbb{Z}_p$, còn có những trường hữu hạn khác: với *mọi* số nguyên tố p và số nguyên $n \ge 1$, tồn tại *duy nhất* một trường có đúng p^n phần tử, gọi là **trường Galois** bậc p^n, ký hiệu $GF(p^n)$.

Ghi chú cho người học — trường hữu hạn là nền tảng của bảo mật hiện đại

Đây là chỗ đại số tuyến tính gặp trực tiếp ngành của bạn. Trường hữu hạn có mặt ở:

- **AES** (chuẩn mã hóa hiện tại): mọi phép toán trong AES diễn ra trên $GF(2^8)$ — mỗi byte là một phần tử của trường có 256 phần tử. Bước `MixColumns` chính là *nhân ma trận* trên $GF(2^8)$.
- **Đường cong elliptic (ECC)**: dùng trong TLS, Bitcoin, chữ ký số. Toàn bộ số học diễn ra trên $\mathbb{Z}_p$ với p là số nguyên tố rất lớn.
- **RSA**: dựa trên số học modulo $n = pq$.
- **Chia sẻ bí mật Shamir** (đã nhắc ở Mục 6.5): nội suy đa thức trên $\mathbb{Z}_p$.
- **Mã Reed–Solomon**: QR code, CD/DVD, RAID-6, lưu trữ đám mây — đều trên $GF(2^8)$.
- **Hàm băm và checksum**: CRC chính là phép chia đa thức trên $GF(2)$.

Vì sao trường hữu hạn lại tiện đến thế: không có sai số làm tròn. Trong $\mathbb{R}$, câu hỏi "số này có bằng 0 không?" luôn mập mờ vì dấu chấm động; trong $\mathbb{Z}_p$ thì câu trả lời luôn chính xác tuyệt đối. Mọi định lý về hạng, cơ sở, khả nghịch đều đúng *chính xác*, không cần lo ngưỡng sai số. Đó là lý do mã sửa lỗi hoạt động hoàn hảo còn tính toán số thực thì luôn phải cẩn thận.
Lưu ý quan trọng: $GF(2^8)$ *không phải* $\mathbb{Z}_{256}$! Vì 256 không nguyên tố, $\mathbb{Z}_{256}$ không là trường. $GF(2^8)$ được xây bằng đa thức trên $GF(2) = \mathbb{Z}_2$ modulo một đa thức bất khả quy bậc 8. Đây là nhầm lẫn rất phổ biến khi mới học.

Mã sửa lỗi

Cho F là trường hữu hạn. Với $n \ge 1$, F^n là không gian vector các hàng $1 \times n$ trên F. Các hàng trong F^n gọi là **từ** (word), viết gọn $[a\ b\ c\ d] = abcd$; mỗi thành phần gọi là một *chữ số* (digit).

Một tập con khác rỗng $C \subseteq F^n$ gọi là một **mã** (code), và các phần tử của C gọi là **từ mã** (code words). Nếu $F = \mathbb{Z}_2$, đó là *mã nhị phân*.

Khoảng cách Euclid không dùng được ở đây (trong $(\mathbb{Z}_2)^4$, từ $w = 1111$ có "khoảng cách bình phương" tới 0 bằng $1+1+1+1 = 0$!). Thay vào đó ta dùng khoảng cách của Richard Hamming.

Trọng số và khoảng cách Hamming

Với từ $\mathbf{w} = a_1 a_2 \cdots a_n$ trong F^n, **trọng số Hamming** là số chữ số khác không:

$$\mathrm{wt}(\mathbf{w}) = |\{i \mid a_i \neq 0\}|$$

Với từ khác $\mathbf{v} = b_1 \cdots b_n$, **khoảng cách Hamming** là

$$d(\mathbf{v}, \mathbf{w}) = \mathrm{wt}(\mathbf{v} - \mathbf{w}) = |\{i \mid b_i \neq a_i\}|$$

tức *số vị trí mà hai từ khác nhau*.

Định lý 8.8.2

Với các từ $\mathbf{u}, \mathbf{v}, \mathbf{w}$ trong F^n:

1. $d(\mathbf{v}, \mathbf{w}) \geq 0$;
2. $d(\mathbf{v}, \mathbf{w}) = 0$ khi và chỉ khi $\mathbf{v} = \mathbf{w}$;
3. $d(\mathbf{v}, \mathbf{w}) = d(\mathbf{w}, \mathbf{v})$;
4. $d(\mathbf{v}, \mathbf{w}) \leq d(\mathbf{v}, \mathbf{u}) + d(\mathbf{u}, \mathbf{w})$.

Vậy d đúng là một *metric* (so với Định lý 5.3.3). Với $r > 0$, định nghĩa **quả cầu**

$$B_r(\mathbf{w}) = \{\mathbf{x} \in F^n \mid d(\mathbf{w}, \mathbf{x}) \leq r\}$$

Giải mã theo láng giềng gần nhất

Cho C là một mã. Nếu từ $\mathbf{v}$ được gửi và $\mathbf{w}$ được nhận, thì $\mathbf{w}$ được giải mã thành *từ mã trong C gần nó nhất*. (Nếu có nhiều từ mã cùng khoảng cách, chọn tùy ý.)

Định lý 8.8.3

Cho C là mã với **khoảng cách tối thiểu** d (khoảng cách nhỏ nhất giữa hai từ mã khác nhau). Khi đó:

1. C **phát hiện** được tối đa t lỗi nếu $d \geq t + 1$;
2. C **sửa** được tối đa t lỗi nếu $d \geq 2t + 1$.

Ghi chú cho người học — vì sao $2t+1$ để sửa nhưng chỉ $t+1$ để phát hiện

Đây là ý tưởng hình học đẹp nhất của cả mục, và rất đáng hình dung.
Tưởng tượng các từ mã là những *điểm* trong không gian F^n, cách nhau ít nhất d. Quanh mỗi từ mã, vẽ quả cầu bán kính t.
Phát hiện t lỗi: chỉ cần các quả cầu bán kính t *không chứa* từ mã nào khác. Nếu $d \geq t+1$, một từ bị lỗi $\leq t$ chỗ chắc chắn *không phải* từ mã hợp lệ $\Rightarrow$ ta biết có lỗi. Nhưng không biết từ gốc là gì.

Sửa t lỗi: cần các quả cầu bán kính t *rời nhau hoàn toàn*. Hai quả cầu bán kính t quanh hai điểm cách nhau d rời nhau khi $d > 2t$, tức $d \geq 2t+1$. Khi đó từ nhận được nằm trong đúng *một* quả cầu $\Rightarrow$ ta biết chắc từ mã gốc.
Ví dụ cụ thể:

- **Bit chẵn lẻ** (parity bit) có $d = 2$: phát hiện được 1 lỗi ($2 \geq 1+1$), sửa được 0 lỗi ($2 < 2 \cdot 1 + 1 = 3$). Đây là lý do parity bit chỉ báo lỗi chứ không sửa được.
- **Mã lặp 3 lần** ($0 \to 000$, $1 \to 111$) có $d = 3$: sửa được 1 lỗi ($3 \geq 3$) bằng "bỏ phiếu đa số".
- **Mã Hamming (7,4)** có $d = 3$: sửa 1 lỗi, nhưng chỉ tốn 3 bit dư thay vì 2 bit như mã lặp — hiệu quả hơn nhiều.

Đánh đổi cơ bản của lý thuyết mã: muốn d lớn (sửa nhiều lỗi) thì phải *hy sinh* số từ mã (tức giảm lượng thông tin truyền được). Tìm điểm cân bằng tối ưu là bài toán trung tâm của ngành này.

Mã tuyến tính

Mã tuyến tính

Một mã $C \subseteq F^n$ gọi là **tuyến tính** nếu nó là một *không gian con* của F^n. Nếu $\dim C = k$, ta gọi C là một (n,k)**-mã**.

Định lý 8.8.5

Cho C là mã tuyến tính với khoảng cách tối thiểu d. Khi đó:

1. $d = \min\{\mathrm{wt}(\mathbf{c}) \mid \mathbf{c} \in C,\ \mathbf{c} \neq \mathbf{0}\}$ — tức khoảng cách tối thiểu bằng *trọng số nhỏ nhất của từ mã khác không*;
2. C phát hiện được t lỗi khi và chỉ khi $d \geq t+1$;
3. C sửa được t lỗi khi và chỉ khi $d \geq 2t+1$;
4. Nếu $|F| = q$ thì $|C| = q^k$.

Vì sao tính tuyến tính làm mọi thứ dễ hơn hẳn

Phần (1) của Định lý 8.8.5 là lý do người ta *luôn* dùng mã tuyến tính trong thực tế.
Với mã bất kỳ có M từ mã, tìm khoảng cách tối thiểu đòi hỏi so sánh $\binom{M}{2} \approx M^2/2$ cặp.
Với mã tuyến tính, chỉ cần tìm *trọng số nhỏ nhất* trong $M-1$ từ mã khác không — giảm từ $O(M^2)$ xuống $O(M)$.
Lý do: vì C là không gian con, hiệu của hai từ mã *vẫn là từ mã*. Nên $d(\mathbf{c}_1, \mathbf{c}_2) = \mathrm{wt}(\mathbf{c}_1 - \mathbf{c}_2)$ luôn là trọng số của một từ mã nào đó.
Tính tuyến tính còn cho ba lợi ích lớn khác: mã hóa chỉ là *một phép nhân ma trận*; toàn bộ mã lưu được bằng k vector cơ sở thay vì q^k từ mã; và giải mã dùng *hội chứng* (dưới đây) rất nhanh.

Ma trận sinh và ma trận kiểm tra chẵn lẻ

Nếu C là (n,k)-mã với cơ sở $\{\mathbf{w}_1,\ldots,\mathbf{w}_k\}$, đặt G là ma trận $k\times n$ có các hàng là $\mathbf{w}_i$. Khi đó

$$C=\{\mathbf{u}G\mid \mathbf{u}\in F^k\}$$

G gọi là **ma trận sinh** (generator matrix) của C, và $\operatorname{rank}G=k$.

Mọi mã tuyến tính đều có ma trận sinh dạng **chuẩn tắc** (systematic):

$$G=\begin{bmatrix} I_k & A\end{bmatrix}$$

với A cỡ $k\times(n-k)$. Khi đó $\mathbf{u}G=[\,\mathbf{u}\;\;\mathbf{u}A\,]$ — tức k chữ số đầu của từ mã *chính là* thông điệp gốc, và $n-k$ chữ số cuối là các **chữ số kiểm tra** (parity digits). Việc khôi phục $\mathbf{u}$ từ $\mathbf{u}G$ trở nên cực kỳ đơn giản.

Ma trận kiểm tra chẵn lẻ

Nếu C là (n,k)-mã, ma trận $(n-k)\times n$ H gọi là **ma trận kiểm tra chẵn lẻ** (parity-check matrix) của C nếu

$$C=\{\mathbf{w}\in F^n\mid H\mathbf{w}^T=\mathbf{0}\}$$

Nếu $G=[\,I_k\;\;A\,]$ là ma trận sinh chuẩn tắc, thì

$$H=\begin{bmatrix}-A^T & I_{n-k}\end{bmatrix}$$

là một ma trận kiểm tra chẵn lẻ của C.

Giải mã bằng hội chứng

Giả sử từ mã $\mathbf{c}$ được gửi, từ $\mathbf{v}$ được nhận, và $\mathbf{z}=\mathbf{v}-\mathbf{c}$ là *vector lỗi*. Bên nhận tính

$$\mathbf{s}=H\mathbf{v}^T$$

gọi là **hội chứng** (syndrome). Vì $H\mathbf{c}^T=\mathbf{0}$, ta có

$$\mathbf{s}=H\mathbf{v}^T=H(\mathbf{c}+\mathbf{z})^T=H\mathbf{z}^T$$

tức *hội chứng chỉ phụ thuộc vào lỗi, không phụ thuộc thông điệp.* Điều này thu hẹp việc tìm kiếm từ q^n khả năng xuống còn q^{n-k} — đó là **giải mã bằng hội chứng** (syndrome decoding).

Định lý 8.8.7

Cho C là (n,k)-mã với ma trận sinh chuẩn tắc $G=[\,I_k\;\;A\,]$ và ma trận kiểm tra $H=[\,-A^T\;\;I_{n-k}\,]$. Khi đó:

1. H là ma trận sinh của *mã đối ngẫu* $C^{\perp}$;
2. G là ma trận kiểm tra chẵn lẻ của $C^{\perp}$;
3. $\dim C^{\perp}=n-k$.

Mã Hamming $(7,4)$

Xét $F=\mathbb{Z}_2$ với ma trận sinh và ma trận kiểm tra tương ứng

$$G=\begin{bmatrix}1&0&0&0&0&1&1\\0&1&0&0&1&0&1\\0&0&1&0&1&1&0\\0&0&0&1&1&1&1\end{bmatrix},\qquad H=\begin{bmatrix}0&1&1&1&1&0&0\\1&0&1&1&0&1&0\\1&1&0&1&0&0&1\end{bmatrix}$$

Mã $C=\{\mathbf{u}G\mid\mathbf{u}\in F^4\}$ có số chiều $k=4$ và gọi là **mã Hamming** $(7,4)$. Nó có khoảng cách tối thiểu $d=3$, nên *phát hiện được 2 lỗi và sửa được 1 lỗi* theo Định lý 8.8.5.

Ghi chú cho người học — giải mã hội chứng trong thực tế, và mã Hamming

Vì sao giải mã hội chứng nhanh: với mã Hamming $(7,4)$, hội chứng $\mathbf{s}=H\mathbf{v}^T$ là một vector 3 bit, tức có 8 giá trị. Nếu $\mathbf{s}=\mathbf{0}$: không có lỗi. Nếu $\mathbf{s}\neq\mathbf{0}$: hội chứng *chính bằng cột thứ i của H*, trong đó i là *vị trí bit bị lỗi*! Chỉ cần lật bit đó là xong.
Đây là điều đáng kinh ngạc: bạn sửa lỗi bằng *một phép nhân ma trận và một phép tra bảng*, không cần so sánh với 16 từ mã. Đó chính là sức mạnh của tính tuyến tính.
Mã Hamming ở đâu trong đời thật:

- **RAM ECC** trong máy chủ: dùng mã Hamming mở rộng (SECDED — Single Error Correct, Double Error Detect) để sửa lỗi bit do tia vũ trụ. Đây là lý do máy chủ dùng RAM ECC còn máy bàn thì không.
- **Bộ nhớ flash và SSD**: mọi ổ SSD đều có bộ điều khiển chạy mã sửa lỗi (thường là LDPC hoặc BCH, các mã mạnh hơn nhưng cùng nguyên lý).
- **Truyền thông vệ tinh và không dây**: 5G dùng LDPC và polar codes.

Bài tập thực hành gợi ý cho bạn: viết một chương trình Python cài đặt mã Hamming $(7,4)$ — mã hóa 4 bit thành 7 bit, cố tình lật một bit ngẫu nhiên, rồi dùng hội chứng để tìm và sửa. Toàn bộ số học trên $\mathbb{Z}_2$ chính là phép `XOR`, nên code rất ngắn. Đây là bài tập nối trực tiếp đại số tuyến tính với lập trình hệ thống, và rất đáng làm.

8.9 Ứng dụng: Dạng toàn phương

Biểu thức như $x_1^2+x_2^2+x_3^2-2x_1x_3+x_2x_3$ gọi là một *dạng toàn phương* theo các biến x_1,x_2,x_3. Mục này chứng minh rằng luôn tìm được biến mới y_1,y_2,y_3 sao cho dạng toàn phương, khi viết theo biến mới, *không còn số hạng chéo*.

Định nghĩa 8.21 — Dạng toàn phương

Một **dạng toàn phương** q theo n biến $x_1,\dots,x_n$ là tổ hợp tuyến tính của các số hạng $x_1^2,\dots,x_n^2$ và các số hạng chéo $x_1x_2,x_1x_3,\dots$

Mọi dạng toàn phương viết gọn được thành tích ma trận

$$q=q(\mathbf{x})=\mathbf{x}^TA\mathbf{x}$$

với $\mathbf{x}=(x_1,\dots,x_n)^T$ và A là ma trận thực $n\times n$. Chú ý: với $i\neq j$, ta có thể thay $a_{ij}x_ix_j$ và $a_{ji}x_jx_i$ bằng $\frac{1}{2}(a_{ij}+a_{ji})x_ix_j$ và $\frac{1}{2}(a_{ij}+a_{ji})x_jx_i$ mà không đổi giá trị dạng. Vậy *không mất tính tổng quát khi giả sử A đối xứng*.

Ví dụ 8.9.1

Viết $q = x_1^2 + 3x_3^2 + 2x_1x_2 - x_1x_3$ dưới dạng $q(\mathbf{x}) = \mathbf{x}^T A\mathbf{x}$ với A đối xứng 3×3.
Lời giải. Các số hạng chéo: $2x_1x_2 = x_1x_2 + x_2x_1$ và $-x_1x_3 = -\frac{1}{2}x_1x_3 - \frac{1}{2}x_3x_1$. Hệ số của x_2x_3, x_3x_2 và x_2^2 đều bằng 0. Vậy

$$q(\mathbf{x}) = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 1 & 1 & -\frac{1}{2} \\ 1 & 0 & 0 \\ -\frac{1}{2} & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Định lý 8.9.1 — Định lý chéo hóa dạng toàn phương

Cho $q = \mathbf{x}^T A\mathbf{x}$ là dạng toàn phương với A đối xứng $n \times n$. Cho P là ma trận trực giao sao cho $P^T AP$ chéo, và định nghĩa biến mới

$$\mathbf{x} = P\mathbf{y} \qquad \text{tương đương} \qquad \mathbf{y} = P^T\mathbf{x}$$

Khi biểu diễn q theo các biến mới, kết quả là

$$q = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2$$

trong đó $\lambda_1, \ldots, \lambda_n$ là các trị riêng của A (kể cả bội).

Chứng minh. Thế $\mathbf{x} = P\mathbf{y}$:

$$q = (P\mathbf{y})^T A(P\mathbf{y}) = \mathbf{y}^T(P^T AP)\mathbf{y} = \mathbf{y}^T D\mathbf{y} = \lambda_1 y_1^2 + \cdots + \lambda_n y_n^2$$

■

Tập vector riêng trực chuẩn $\{\mathbf{f}_1, \ldots, \mathbf{f}_n\}$ tương ứng gọi là tập **trục chính** của dạng q. Vì $\mathbf{x} = y_1\mathbf{f}_1 + \cdots + y_n\mathbf{f}_n$ và $y_i = \mathbf{x} \cdot \mathbf{f}_i$ (Định lý khai triển), ta có công thức gọn:

$$q(\mathbf{x}) = \lambda_1(\mathbf{x} \cdot \mathbf{f}_1)^2 + \cdots + \lambda_n(\mathbf{x} \cdot \mathbf{f}_n)^2$$

Ghi chú cho người học — dấu của trị riêng quyết định hình dạng

Sau khi chéo hóa, $q = \lambda_1 y_1^2 + \cdots + \lambda_n y_n^2$ và mọi tính chất hình học của q đọc được ngay từ *dấu* các λ_i:

Dấu các λ_i	**Loại dạng**	**Hình dạng ($n = 2$)**
Tất cả > 0	Xác định dương	Ellipse (bát ngửa)
Tất cả < 0	Xác định âm	Ellipse (bát úp)
Trái dấu	Không xác định	Hyperbol (yên ngựa)
Có $\lambda_i = 0$	Suy biến	Parabol / đường thẳng

Ứng dụng trong ngành của bạn:

- **Tối ưu hóa:** hàm mất mát gần điểm dừng xấp xỉ bằng dạng toàn phương với ma trận Hessian. Chéo hóa cho biết "thung lũng" hẹp theo hướng nào — tỉ số λ_1/λ_n quyết định gradient descent hội tụ nhanh hay chậm (đó là lý do người ta dùng momentum và Adam).
- **Thống kê:** khoảng cách Mahalanobis $q = (\mathbf{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})$ là dạng toàn phương; các mặt $q = c$ là các ellipsoid tin cậy.
- **Đồ họa:** ellipsoid bao (bounding ellipsoid) trong phát hiện va chạm.
- **Vật lý:** tensor quán tính, tensor ứng suất — trục chính chính là các trục quán tính chính của vật thể.

8.10 Ứng dụng: Tối ưu hóa có ràng buộc

Trong ứng dụng, ta thường phải làm cực đại hoặc cực tiểu một hàm $q(x_1,\dots,x_n)$ (gọi là *hàm mục tiêu*) trên một *miền khả thi* nào đó trong $\mathbb{R}^n$. Ở đây ta xét trường hợp quan trọng khi q là một dạng toàn phương.

Ví dụ 8.10.1

Một chính trị gia đề xuất chi x_1 đô-la mỗi năm cho y tế và x_2 đô-la cho giáo dục, với ràng buộc ngân sách

$$5x_1^2 + 3x_2^2 \le 15$$

Bà muốn chọn (x_1,x_2) để cực đại hóa một thước đo mức hài lòng của cử tri $q(x_1,x_2) = x_1x_2$. Chia ràng buộc cho 15: $\left(\frac{x_1}{\sqrt{3}}\right)^2 + \left(\frac{x_2}{\sqrt{5}}\right)^2 \le 1$. Đặt $y_1 = \frac{x_1}{\sqrt{3}}$, $y_2 = \frac{x_2}{\sqrt{5}}$, ràng buộc thành $\|\mathbf{y}\| \le 1$, và hàm mục tiêu thành $q = \sqrt{15}\,y_1y_2$.

Vậy nói chung, không mất tính tổng quát khi giả sử ràng buộc là $\|\mathbf{x}\| \le 1$. Khi đó Định lý trục chính giải trọn bài toán.

Định lý 8.10.1

Xét dạng toàn phương $q(\mathbf{x}) = \mathbf{x}^TA\mathbf{x}$ với A đối xứng $n\times n$, và gọi λ_1, λ_n là trị riêng *lớn nhất* và *nhỏ nhất* của A. Khi đó:

1. $\max\{q(\mathbf{x}) \mid \|\mathbf{x}\| \le 1\} = \lambda_1$, và $q(\mathbf{f}_1) = \lambda_1$ với $\mathbf{f}_1$ là vector riêng đơn vị bất kỳ ứng với λ_1;
2. $\min\{q(\mathbf{x}) \mid \|\mathbf{x}\| \le 1\} = \lambda_n$, tương tự với $\mathbf{f}_n$.

Chứng minh. Sắp xếp $\lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_n$. Theo Định lý trục chính, lấy P trực giao với $P^TAP = D$. Đặt $\mathbf{y} = P^T\mathbf{x}$; khi đó $\|\mathbf{y}\| = \|\mathbf{x}\|$ (vì P trực giao bảo toàn độ dài) và

$$q(\mathbf{x}) = \lambda_1y_1^2 + \lambda_2y_2^2 + \cdots + \lambda_ny_n^2 \tag{8.9}$$

Giả sử $\|\mathbf{x}\| \le 1$. Vì $\lambda_i \le \lambda_1$ với mọi i:

$$q(\mathbf{x}) \le \lambda_1(y_1^2+\cdots+y_n^2) = \lambda_1\|\mathbf{y}\|^2 \le \lambda_1$$

Cực đại này đạt được tại $\mathbf{x} = \mathbf{f}_1$. Chứng minh cho cực tiểu tương tự. ■

Ghi chú cho người học — Định lý 8.10.1 là nền của rất nhiều thuật toán

Định lý 8.10.1 phát biểu một sự kiện cực kỳ hữu dụng:

Trị riêng lớn nhất là giá trị lớn nhất của $\mathbf{x}^T A\mathbf{x}$ trên hình cầu đơn vị, và nó đạt được tại vector riêng tương ứng.

Đại lượng $\dfrac{\mathbf{x}^T A\mathbf{x}}{\mathbf{x}^T\mathbf{x}}$ gọi là **thương Rayleigh** (đã gặp ở Mục 8.5). Định lý này nói: cực đại của thương Rayleigh = λ_1, cực tiểu = λ_n.
Hệ quả trực tiếp:

- **PCA** (Mục 8.11): "tìm hướng giữ nhiều phương sai nhất" chính là bài toán cực đại $\mathbf{x}^T\Sigma\mathbf{x}$ với $\|\mathbf{x}\| = 1$ — câu trả lời là vector riêng ứng với trị riêng lớn nhất. *Đây là toàn bộ nội dung toán học của PCA.*
- **Chuẩn ma trận:** $\|A\|_2 = \sigma_1 = \sqrt{\lambda_1(A^TA)}$ — hệ số giãn lớn nhất.
- **Spectral clustering:** bài toán cắt đồ thị tối ưu, sau khi nới lỏng, trở thành bài toán trị riêng của ma trận Laplace.
- **Phương pháp lũy thừa:** giải thích vì sao lặp $\mathbf{x} \leftarrow A\mathbf{x}$ hội tụ về hướng làm thương Rayleigh lớn nhất.

Một cách diễn đạt khác đáng nhớ: *bài toán tối ưu hóa dạng toàn phương trên hình cầu không cần thuật toán lặp — nó có nghiệm dạng đóng, chính là bài toán trị riêng.* Đây là một trong số ít bài toán tối ưu phi lồi mà ta giải được chính xác.

8.11 Ứng dụng: Phân tích thành phần chính (PCA)

Đại số tuyến tính rất quan trọng trong phân tích đa biến của thống kê. Ta kết thúc chương bằng một ứng dụng ngắn của chéo hóa trong lĩnh vực này.

Biến ngẫu nhiên X là hàm nhận giá trị thực theo một luật xác suất. Giá trị của X phân bố quanh một số trung tâm $\mu = E(X)$ gọi là *kỳ vọng*. **Phương sai** của X là

$$\operatorname{var}(X) = E\{(X-\mu)^2\}$$

và $\sigma = \sqrt{\operatorname{var}(X)}$ là *độ lệch chuẩn*.

Với hai biến ngẫu nhiên X, Y, **hiệp phương sai** là

$$\operatorname{cov}(X,Y) = E\{(X-\mu)(Y-\nu)\}, \qquad \mu = E(X),\ \nu = E(Y)$$

Rõ ràng $\operatorname{cov}(X,X) = \operatorname{var}(X)$. Nếu $\operatorname{cov}(X,Y) = 0$ thì X và Y gọi là *không tương quan*.

Với họ biến ngẫu nhiên $X_1, \dots, X_n$, đặt $\sigma_{ij} = \operatorname{cov}(X_i, X_j)$. Khi đó **ma trận hiệp phương sai** là ma trận $n \times n$

$$\Sigma = [\sigma_{ij}]$$

Ma trận Σ rõ ràng *đối xứng*; hơn nữa nó *nửa xác định dương* (mọi trị riêng $\lambda \geq 0$).

Giả sử các trị riêng của Σ là $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$. Định lý trục chính cho ma trận trực giao P với

$$P^T\Sigma P = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$$

Đặt $\mathbf{X} = (X_1, \dots, X_n)$ và định nghĩa biến mới

$$\mathbf{Y} = P^T\mathbf{X}$$

Các biến ngẫu nhiên $Y_1, \ldots, Y_n$ gọi là các **thành phần chính** (principal components) của các X_i; chúng là tổ hợp tuyến tính của các X_i. Hơn nữa, có thể chứng minh

$$\text{cov}(Y_i, Y_j) = 0 \text{ nếu } i \neq j, \qquad \text{var}(Y_i) = \lambda_i$$

Tổng phương sai của một tập biến ngẫu nhiên gọi là **phương sai toàn phần**. Vì Σ và ma trận chéo đồng dạng nên chúng có cùng *vết*:

$$\sigma_{11} + \sigma_{22} + \cdots + \sigma_{nn} = \lambda_1 + \lambda_2 + \cdots + \lambda_n$$

Nghĩa là các thành phần chính Y_i có *cùng phương sai toàn phần* với các X_i ban đầu. Nhưng vì $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$, phần lớn phương sai đó tập trung ở vài Y_i *đầu tiên*.

Trong thực hành, các nhà thống kê thấy rằng nghiên cứu vài Y_i đầu (và bỏ qua phần còn lại) cho phân tích khá chính xác về biến thiên toàn hệ thống. Điều này dẫn tới **giảm chiều dữ liệu** đáng kể. Hơn nữa, phân tích các thành phần chính thường *phát lộ* những quan hệ giữa các X_i mà trước đó không ai ngờ tới.

Ghi chú cho người học — PCA từ đầu đến cuối, bằng ngôn ngữ của bạn

PCA là ứng dụng nổi tiếng nhất của cả cuốn sách này. Giờ bạn đã có đủ công cụ để hiểu nó *hoàn toàn*. Quy trình:

1. **Quy tâm dữ liệu.** Trừ trung bình mỗi cột (Mục 5.7). Bước này bắt buộc — PCA không quy tâm cho kết quả sai hoàn toàn.
2. **Tính ma trận hiệp phương sai** $\Sigma = \frac{1}{n-1} X_c^T X_c$. Nó đối xứng và nửa xác định dương (Mục 8.3).
3. **Chéo hóa trực giao** Σ (Định lý 8.2.2 — đây là bước cốt lõi, và *bảo đảm* thực hiện được vì Σ đối xứng).
4. **Sắp xếp** trị riêng giảm dần. Vector riêng thứ i là thành phần chính thứ i; trị riêng λ_i là phương sai theo hướng đó.
5. **Giữ k thành phần đầu.** Tỉ lệ phương sai giữ lại là $\dfrac{\lambda_1 + \cdots + \lambda_k}{\lambda_1 + \cdots + \lambda_n}$. Thường chọn k nhỏ nhất sao cho tỉ lệ này $\geq 95\%$.
6. **Chiếu dữ liệu** lên k hướng đó (Mục 8.1) — được biểu diễn k chiều thay vì n chiều.

Mỗi bước dùng một kết quả bạn đã học: quy tâm (5.7), đối xứng $\Rightarrow$ chéo hóa trực giao (8.2), trị riêng lớn nhất = phương sai lớn nhất (8.10), chiếu (8.1). Đó là lý do tôi nói Chương 8 là chương quan trọng nhất.
Trong thực hành — đừng dùng ma trận hiệp phương sai: như đã cảnh báo ở Mục 8.4 và 8.6, lập $X^T X$ bình phương số điều kiện. Cách đúng là chạy **SVD** trực tiếp trên dữ liệu đã quy tâm: $X_c = U\Sigma V^T$; khi đó các cột của V chính là thành phần chính, và $\lambda_i = \sigma_i^2/(n-1)$. Đây chính là cách `sklearn.decomposition.PCA` cài đặt.
Giới hạn của PCA cần biết:

- PCA chỉ tìm cấu trúc *tuyến tính*. Dữ liệu nằm trên một mặt cong (manifold) sẽ bị PCA bỏ sót — khi đó dùng kernel PCA, t-SNE hay UMAP.
- PCA *nhạy với thang đo*. Nếu một đặc trưng đo bằng mét và một bằng milimét, đặc trưng thứ hai sẽ chi phối hoàn toàn. Luôn chuẩn hóa (standardize) trước, trừ khi các đặc trưng đã cùng đơn vị.
- Thành phần chính *khó diễn giải* — chúng là tổ hợp của mọi đặc trưng gốc. Nếu cần diễn giải, cân nhắc Sparse PCA hoặc NMF.

- PCA tối ưu cho việc *tái tạo* dữ liệu, không phải cho *phân loại*. Hướng giữ nhiều phương sai nhất chưa chắc là hướng phân biệt lớp tốt nhất — khi đó dùng LDA.

Tổng kết Chương 8

Mục	Khái niệm cốt lõi	Điểm mấu chốt
8.1	Gram–Schmidt, phép chiếu	Cơ sở trực giao luôn tồn tại; $\dim U + \dim U^{\perp} = n$
8.2	Chéo hóa trực giao	**Đối xứng $\iff$ chéo hóa trực giao được** (Định lý phổ)
8.3	Xác định dương, Cholesky	$\mathbf{x}^T A\mathbf{x} > 0$; $A = U^T U$; "cái bát ngửa"
8.4	Phân rã QR	Cách *đúng* để giải bình phương tối thiểu
8.5	Tính trị riêng	Phương pháp lũy thừa (PageRank) và thuật toán QR
8.6	**SVD**	$A = P\Sigma Q^T$ luôn tồn tại; quay–giãn–quay; nền của PCA, nén, A^+
8.7	Ma trận phức	Hermite $\leftrightarrow$ đối xứng; unita $\leftrightarrow$ trực giao
8.8	Mã tuyến tính	Khoảng cách Hamming; $d \geq 2t+1$ để sửa t lỗi; giải mã hội chứng
8.9	Dạng toàn phương	Chéo hóa để khử số hạng chéo; dấu λ_i = hình dạng
8.10	Tối ưu có ràng buộc	$\max \mathbf{x}^T A\mathbf{x}$ trên cầu đơn vị $= \lambda_1$
8.11	PCA	Chéo hóa ma trận hiệp phương sai $\Rightarrow$ giảm chiều

Một câu tóm tắt cả chương: *Trực giao biến các bài toán khó thành các bài toán trị riêng — và bài toán trị riêng thì máy tính giải được ổn định, nhanh, và chính xác.*
Bốn thứ đáng ghi nhớ nhất cho sự nghiệp IT của bạn:

1. **Định lý phổ** (8.2): ma trận đối xứng luôn chéo hóa trực giao được. Đây là lý do PCA, spectral clustering, và phân tích Hessian hoạt động.
2. **SVD** (8.6): mọi ma trận đều phân rã được. Đây là công cụ vạn năng — hạng, số điều kiện, nén, nghịch đảo giả, PCA.
3. **Đừng lập** $A^T A$ (8.4): dùng QR hoặc SVD. Quy tắc này cứu bạn khỏi rất nhiều lỗi số học.
4. **Mã tuyến tính** (8.8): mọi hệ thống lưu trữ và truyền tin đáng tin cậy đều đứng trên đại số tuyến tính trên trường hữu hạn.

Chương 8 — Tính trực giao — bản dịch tiếng Việt gồm bảng thuật ngữ và toàn bộ 11 mục 8.1–8.11 (định nghĩa, định lý, chứng minh, ví dụ có lời giải).

Phần bài tập (Exercises) của Chương 8 không được dịch trong bản này — xin xem trực tiếp bản gốc tiếng Anh, trang 407–494. Một số chứng minh kỹ thuật dài (Định lý 8.8.5, 8.8.6, các bổ đề phụ của Mục 8.6) được trình bày ở dạng phát biểu kèm ý tưởng; xin đối chiếu bản gốc nếu cần chi tiết đầy đủ.

Bản dịch dựa trên *Linear Algebra with Applications*, W. Keith Nicholson, Lyryx Open Edition (Version

2023 — Revision A), phát hành theo giấy phép **CC BY-NC-SA 4.0**. Bản dịch này là một tác phẩm phái sinh và cũng được phát hành theo cùng giấy phép CC BY-NC-SA 4.0.